Fact checking con evidenza evidenziata — piano operativo

Indice

Fact checking con evidenza evidenziata — piano operativo **3**
1. Che cosa si vuole ottenere . . . 3
2. Come funziona, passo per passo . . . 4
   2.1 Che cosa fa già il decision model . . . 4
   2.2 Che cosa aggiungiamo . . . 4
   2.3 Che cosa entra nella testa . . . 4
   2.4 Che cosa fa la testa . . . 5
   2.5 Perché il preambolo . . . 5
   2.6 Che cosa si addestra . . . 5
3. I modelli . . . 5
4. Glossario . . . 6
5. Sequenza di lavoro . . . 6
6. Obiettivi . . . 7

Passo 1 — Caricare i modelli e leggere vettori e attenzione **7**
In breve . . . 7
1. Pacchetti . . . 8
2. I modelli . . . 8
3. Trovare i layer nel codice del modello . . . 9
4. Hook sui vettori . . . 9
5. Hook sull'attenzione . . . 10
6. Test . . . 10
7. GPU necessaria . . . 11

Passo 2 — Documenti divisi in frasi, con le frasi di evidenza **11**
In breve . . . 11
1. Una riga . . . 11
2. Come si divide in frasi . . . 12
3. FEVER . . . 12
4. HoVer . . . 12
5. SciFact e WiCE (solo per il test, mai per il training) . . . 13
6. MiniCheck C2D e D2C . . . 13
7. Comandi . . . 14
8. Controlli . . . 14

Passo 3 — Altri dati con evidenza nota **14**
In breve . . . 14
1. Documenti generati con posizioni note (C2D e CG2C) . . . 14
2. Etichette ricavate col modello (ablazione) . . . 15
3. Evidenza spostata (righe sparse) . . . 15
4. Quante righe . . . 16
5. Controlli . . . 16

Passo 4 — Split e mix di training **16**
In breve . . . 16
1. Regole degli split . . . 16

2. I set di valutazione . . . 17
3. Il mix `ev_head` (training della sola testa) . . . 17
4. Il mix `ev_joint` (training anche del modello, fase 2) . . . 18

Passo 5 — Il prompt con il preambolo **18**
In breve . . . 18
1. Esempio (KnowLine-4B, schema) . . . 19
2. Dove va il preambolo negli altri modelli . . . 19
3. Scegliere il testo del preambolo . . . 19
4. Riusare il verdetto o riaddestrarlo . . . 20
5. Dove si scrive la scelta . . . 20

Passo 6 — Dalle frasi ai token, e salvataggio dei vettori **20**
In breve . . . 20
1. L'esempio pronto per il modello . . . 20
2. Trovare i token di ogni frase . . . 21
3. Documenti troppo lunghi . . . 21
4. Che cosa si salva . . . 22
5. Comando . . . 22
6. Test . . . 22

Passo 7 — Metodi semplici da battere **23**
In breve . . . 23
1. I metodi . . . 23
2. Come si calcola E-ATT . . . 23
3. Quali frasi si mostrano . . . 23
4. Comandi . . . 24
5. Tabella da compilare (una per modello) . . . 24

Passo 8 — La testa e la loss **24**
In breve . . . 24
1. La testa, passo per passo . . . 24
2. Loss di partenza (H0): per frase, con soglia fissa . . . 25
3. Loss con soglia per riga (quella da provare in H-LOSS) . . . 26
Esempio numerico (anche come test automatico) . . . 26
4. Termine aggiuntivo per le coppie (solo nell'esperimento H-PAIR) . . . 27
5. Test . . . 27

Passo 9 — Training **27**
In breve . . . 27
1. Fase 1a . . . 27
2. Quali frasi si mostrano . . . 28
3. Che cosa guardare durante il training . . . 28
4. Fase 1b . . . 29
5. Fase 2: training anche del modello . . . 29

Passo 10 — Valutazione **29**
In breve . . . 29
1. Su quali righe . . . 30
2. Metriche dell'evidenza . . . 30
3. Metriche del verdetto . . . 30
4. Prove di comportamento . . . 31
5. Latenza . . . 31
6. Comandi . . . 31
7. Report finale (uno per modello) . . . 31

Passo 11 — Esperimenti **32**
In breve . . . 32
Regole . . . 32

Prima degli esperimenti . . . 32
La testa di partenza: H0 . . . 32
Gli esperimenti principali (in quest'ordine) . . . 32
Controlli sulla configurazione migliore . . . 33
Esperimenti da fare solo se serve . . . 33
elephant-1 e gli altri modelli . . . 33
Controlli finali . . . 34
Registro . . . 34

Passo 12 — Server e rilascio **34**
In breve . . . 34
1. Che cosa fa il server per ogni richiesta . . . 34
2. Come gira ogni modello . . . 35
3. La cartella di rilascio . . . 35
4. Controlli prima del rilascio . . . 36
5. Congelamento e test . . . 36

Checklist e problemi frequenti **36**
1. Checklist per passo . . . 36
2. Se succede... . . . 37

Una strada da esplorare: insegnare alla testa quanto ogni frase regge il verdetto **38**
Il problema . . . 38
L'idea . . . 38
Come farlo . . . 39
Quando tenerlo . . . 39
Rischi . . . 39
Perché vale la pena . . . 39

Fact checking con evidenza evidenziata — piano operativo

1. Che cosa si vuole ottenere

Oggi, nel piano binario (../fine_tuning_factcheck/), un decision model riceve un documento e un claim e risponde con una probabilità: P(supportato). Lo fa con **un solo passaggio del modello sul testo** (un *forward*), senza generare parole: per questo è veloce.

Vogliamo che, quando il claim è supportato, il sistema dica anche **quali frasi del documento lo dimostrano**, senza perdere velocità: sempre un solo forward, nessuna parola generata.

Esempio di risposta:

```
{"p_supported": 0.93, "supported": true,
 "evidence": [{"sentence": 0,  "start": 0,    "end": 61,   "score": 0.91},
              {"sentence": 39, "start": 4210, "end": 4262, "score": 0.84}]}
```

- `sentence`: numero della frase nel documento (da 0); `start`/`end`: dove inizia e finisce, in caratteri, nel testo originale.
- Se il claim non è supportato, `evidence` è vuota: non si mostra né cosa manca né cosa lo contraddice.

Il caso difficile è l'**evidenza sparsa**: due frasi lontane che solo **insieme** dimostrano il claim.

> Claim: "Nancy R. Howell teaches at a seminary in Kansas City." Frase 1: "Saint Paul School of Theology is a seminary located in Kansas City, Missouri." … 38 frasi che non c'entrano … Frase 40: "Nancy R. Howell is Professor of Theology at Saint Paul School of Theology."
>
> Nessuna delle due basta da sola. Evidenza corretta: frasi 1 e 40.

2. Come funziona, passo per passo

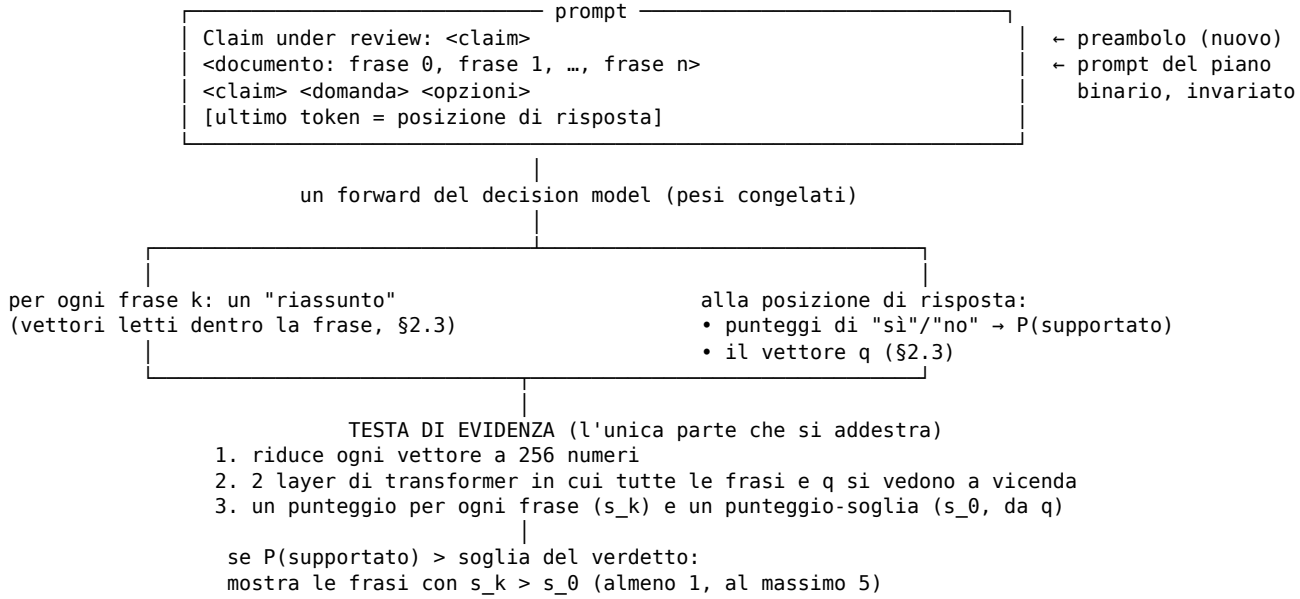
2.1 Che cosa fa già il decision model

Il modello legge il prompt. Per **ogni token** e per **ogni layer** calcola un vettore di numeri (lo *stato nascosto*, lungo 2.560 numeri nei modelli da 4B e 4.096 in quelli da 9B). Nei modelli causali ogni token "vede" solo i token che lo precedono; nei modelli bidirezionali, negli 8 layer ad attenzione piena, vede anche quelli dopo (§3).

Il verdetto si legge dall'**ultimo token del prompt** (la posizione di risposta, chiamata `answer_pos`): lì il modello assegna un punteggio alla risposta "sì" e uno alla risposta "no", e da questi si ricava P(supportato). Questa parte esiste già ed è stata addestrata nel piano binario. **Non la tocchiamo.**

2.2 Che cosa aggiungiamo

Mentre il modello fa quel forward, leggiamo alcuni dei vettori che calcola comunque, e li diamo a una **piccola rete separata** (la *testa di evidenza*, circa 3 milioni di parametri contro i 4–9 miliardi del modello). La testa dà un punteggio a ogni frase.



2.3 Che cosa entra nella testa

Per ogni frase k del documento si leggono, dallo stesso forward:

Nome	Che cos'è	Perché serve
`h_end[k]`	il vettore dell'**ultimo token** della frase	riassume la frase letta fin lì
`h_mean[k]`	la **media** dei vettori di tutti i token della frase	non dipende dal punto finale; tiene conto di tutta la frase
`att[k]`	quanto la posizione di risposta "guarda" i token della frase (pesi di attenzione), negli 8 layer in cui il modello guarda tutto il testo	dice su quali frasi il modello si è basato per decidere

E dalla posizione di risposta:

Nome	Che cos'è
`q`	il vettore della posizione di risposta: ha visto tutto il prompt, quindi "sa" il verdetto e quali frasi servono

I vettori `h_end`, `h_mean` e `q` si leggono da **4 layer** diversi (non solo dall'ultimo) e la testa impara come pesarli.

2.4 Che cosa fa la testa

1. **Comprime** ogni vettore a 256 numeri (una moltiplicazione per una matrice).
2. **Fa comunicare** le frasi: due layer di transformer *non causale*, cioè ogni frase vede tutte le altre e `q`, in avanti e all'indietro. Questo risolve l'evidenza sparsa: nel modello la frase 1 non può vedere la frase 40 (viene dopo: vale per i modelli causali, e per i 24 layer DeltaNet di tutti), nella testa sì.
3. **Dà i punteggi**: `s_k` per ogni frase k, e `s_0` calcolato da `q`, che fa da **soglia di quella riga**. Si mostrano le frasi con `s_k > s_0`. La soglia cambia da documento a documento: se servono 3 frasi può abbassarsi per farle passare tutte.

2.5 Perché il preambolo

Nei modelli causali ogni token vede solo ciò che lo precede. Nel prompt del piano binario il claim viene **dopo** il documento, quindi quando il modello legge le frasi non sa ancora che cosa deve verificare: i vettori delle frasi non dipendono dal claim. Nei modelli bidirezionali le frasi vedono il claim negli 8 layer pieni anche senza preambolo, ma non nei 24 layer DeltaNet, che restano causali: il preambolo aiuta anche lì, ma meno (lo misura l'esperimento H4).

Mettendo una riga `Claim under review: <claim>` **prima** del documento, ogni frase è letta sapendo che cosa si verifica. Il resto del prompt resta identico a quello del piano binario: il claim compare ancora dopo il documento, nel punto in cui il verdetto è stato addestrato a vederlo. Così il verdetto si può riusare (si verifica al passo 05).

2.6 Che cosa si addestra

Solo la testa. Il modello resta congelato, quindi **il verdetto non cambia** e non si rischia di peggiorarlo. La testa impara da documenti in cui si sa quali frasi sono l'evidenza (FEVER, HoVer e dati generati, passi 02–03), e da documenti con claim non supportati, in cui nessuna frase deve superare la soglia.

Siccome il modello è congelato, i suoi vettori si calcolano **una volta sola** e si salvano su disco (06); poi ogni prova di training della testa dura minuti. Solo se questo non basta si addestra anche il modello (con una LoRA), con controlli sul verdetto (09 §6).

3. I modelli

I modelli sono quelli del piano binario (../fine_tuning_factcheck/00_overview.md §2), cioè i 10 decision model dello studio focalizzato. Tutti sono Qwen3.5: 32 layer, di cui 8 ad attenzione piena (3, 7, 11, 15, 19, 23, 27, 31) e 24 Gated DeltaNet, che portano avanti un riassunto del testo letto e restano sempre causali.

Modello	Ruolo	Vettore (d)	Attenzione nei layer pieni
KnowLine-4B	si fanno qui tutte le prove (4B, causale, lettura semplice)	2.560	causale

Modello	Ruolo	Vettore (d)	Attenzione nei layer pieni
elephant-1	conferma a 9B; modello principale del piano binario	4.096	bidirezionale a blocchi
Jeffan-4B, d3-flash, OneJev1.6-9B, Nimble-9B-v3	estensione (codice degli autori in transformers)	2.560 / 4.096	Jeffan, d3: bidirezionale; OneJev, Nimble: causale
StartLux 9B/4B, pplx-decider-flash, Clef-Flash	estensione, con più lavoro (StartLux gira solo con vLLM; pplx e Clef hanno una testa propria)	2.560 / 4.096	StartLux, pplx: bidirezionale; Clef: come caricato

Il verdetto di ogni modello è quello del piano binario (stessa domanda, stessa soglia).

4. Glossario

Termine	Significato
forward	un passaggio del modello su tutto il prompt
stato nascosto, vettore	i numeri che il modello calcola per un token in un layer
posizione di risposta (`answer_pos`)	l'ultimo token del prompt, dove si legge il verdetto
testa di evidenza	la piccola rete aggiunta che dà un punteggio alle frasi
`s_k`, `s_0`	punteggio della frase k; soglia della riga
gold	frasi di evidenza annotate (da persone o note per costruzione)
insieme completo	tutte le frasi gold mostrate (per l'evidenza sparsa conta questo)
sufficienza	le frasi mostrate, da sole come documento, fanno dire "supportato" al modello
fase 1a / 1b / 2	training della testa su vettori salvati / con il modello in memoria / insieme a una LoRA del modello

5. Sequenza di lavoro

#	Passo	File	Prodotto
1	Caricare i modelli e leggere vettori e attenzione	01	`evidence/models.py` verificato
2	Documenti divisi in frasi, con le frasi di evidenza	02	righe da FEVER, HoVer, SciFact, WiCE
3	Altri dati con evidenza nota	03	dati generati, etichette ricavate col modello, evidenza spostata lontano

#	Passo	File	Prodotto
4	Split e mix di training	04	set di valutazione congelati; mix di training
5	Prompt con il preambolo	05	preambolo scelto; verdetto verificato
6	Calcolo e salvataggio dei vettori	06	vettori su disco
7	Metodi semplici di confronto	07	numeri da battere
8	Testa e loss	08	codice testato
9	Training	09	teste addestrate
10	Valutazione	10	metriche e latenza
11	Esperimenti	11	configurazione finale per ogni modello
12	Server e rilascio	12	server che restituisce verdetto ed evidenza
—	Checklist e problemi frequenti	13	
—	Una strada da esplorare	14	

Codice in `evidence/` (pacchetto Python nella radice della repo), dati in `data/evidence/`. Sul server i job GPU si lanciano così (il log va rediretto dentro il comando):

```
gpu-run --batch --name ev-extract-knowline -- bash -c \
  'python -m evidence.extract --model knowline-4b --mix ev_head > logs/ev-extract-knowline.out 2>&1'
```

6. Obiettivi

Obiettivo	Soglia
Verdetto	uguale a quello del piano binario (BAcc e AUC entro 0,3 punti)
F1 sulle frasi	almeno 15 punti sopra il metodo lessicale (E-LEX, 07); indicativo ≥ 0,75 su FEVER e HoVer ≥ 0,65
Insieme completo su HoVer (evidenza su più pagine)	
Evidenza lontana	F1 con frasi a ≥ 20 frasi di distanza al massimo 5 punti sotto quello con frasi vicine
Sufficienza	≥ 85%
Latenza	al massimo +3% rispetto al solo verdetto (modelli 4B: +3% o +2 ms, il maggiore dei due)

Passo 1 — Caricare i modelli e leggere vettori e attenzione

In breve

Serve un modulo, `evidence/models.py`, che per ciascun modello faccia **un forward** su un prompt e restituisca tre cose:

1. i punteggi delle due opzioni nella posizione di risposta → P(supportato), come nel piano binario;
2. alcuni **vettori** (stati nascosti) letti durante il forward: quelli dei token di ogni frase, in 4 layer, e quello della posizione di risposta;
3. per gli 8 layer ad attenzione piena, **quanto la posizione di risposta guarda ciascun token** (una riga di pesi di attenzione).

I punti 2 e 3 si ottengono con degli *hook*: piccole funzioni agganciate ai layer, che copiano i dati mentre il forward passa, senza cambiarlo.

Il forward è quello che il piano binario usa già per il training (`forward_model` del renderer, ../fine_tuning_factcheck/03_rendering_formato.md §4): stesso prompt, stessa maschera, stessa lettura degli autori. Qui si aggiungono solo gli hook.

1. Pacchetti

Lo stesso ambiente del piano binario, più:

```
pip install nltk rank_bm25 safetensors sentence-transformers
python -c "import nltk; nltk.download('punkt_tab')"
```

`flash-linear-attention` e `causal-conv1d` (per i layer DeltaNet) sono già richiesti dal piano binario.

2. I modelli

Tutti Qwen3.5, con la stessa struttura:

	Gruppo 4B	Gruppo 9B
Layer	32	32
Layer ad attenzione piena	3, 7, 11, 15, 19, 23, 27, 31	idem
Altri layer	24 Gated DeltaNet, sempre causali	idem
Teste di query / teste KV (layer pieni)	16 / 4	16 / 4
Lunghezza del vettore (d)	2.560	4.096
4 layer da cui leggere i vettori (partenza)	19, 23, 27, 31	19, 23, 27, 31

I 4 layer di partenza sono gli ultimi 4 ad attenzione piena (il 31 è anche l'ultimo layer). L'esperimento H2a (11) sceglie quali servono davvero.

Che cosa cambia da un modello all'altro, per l'estrazione:

Modello	Forward con hook	Posizione di risposta	Note
KnowLine-4B	transformers (front end degli autori, backend `hf`)	ultimo token del prompt	richieste fino a 3.000 token scritte due volte: le frasi si leggono dalla **seconda** copia, che ha già visto tutto il prompt della prima
elephant-1	motore `qev` in transformers (legge la maschera a blocchi da `modeling_overrides`)	ultimo token del suffisso della domanda	prefisso (stato) e suffisso (domanda): le frasi stanno nel prefisso
Jeffan-4B, d3-flash	codice degli autori in transformers (maschera bidirezionale per tipo di layer)	ultimo token	

Modello	Forward con hook	Posizione di risposta	Note
OneJev1.6-9B, Nimble-9B-v3	transformers con il prompt degli autori, causale	ultimo token della domanda	il loro runtime (Flide, motore Nimble) divide prefisso e domanda: in transformers si usa una sequenza unica e si verifica che il verdetto coincida
StartLux 9B/4B	transformers con la maschera bidirezionale ricostruita (come nel training del piano binario)	ultimo token	gli autori usano solo vLLM: il verdetto va verificato contro il loro server
pplx-decider-flash	pacchetto `pplx_decider`, percorso di riferimento	la posizione di lettura della domanda	`q` = stato nella posizione di lettura (quella che la sua testa legge)
Clef-Flash	`joint_schema_model.py`	ultimo token prima della testa congiunta	`q` = stato finale in quella posizione

3. Trovare i layer nel codice del modello

I nomi interni dei moduli possono cambiare (modelli caricati come multimodali o solo testo). Si cerca la lista dei layer testuali per lunghezza e si leggono i tipi di layer dal config:

```
def text_layers(model, n_layers=32):
    found = [m for name, m in model.named_modules()
             if isinstance(m, torch.nn.ModuleList) and len(m) == n_layers and "visual" not in name and "vision"
              ↪  not in name]
    assert len(found) == 1
    return found[0]

def full_attention_layers(config):
    tc = getattr(config, "text_config", config)
    return [i for i, t in enumerate(tc.layer_types) if t == "full_attention"]      # [3, 7, …, 31]
```

4. Hook sui vettori

Per ognuno dei 4 layer, un hook prende l'uscita del layer (un vettore per token) e calcola subito, sulla GPU:

- il vettore della posizione di risposta (`q`);
- il vettore dell'ultimo token di ogni frase (`h_end`);
- la media dei vettori dei token di ogni frase (`h_mean`).

Non si conservano mai i vettori di tutti i token: solo questi.

```
class StateTap:
    def __init__(self, layers, mix_idx):
        self.out, self.spans, self.answer_pos = {}, None, None   # spans: [n, 2] = primo e ultimo token di ogni
         ↪  frase
        self.handles = [layers[i].register_forward_hook(self._hook(i)) for i in mix_idx]

    def _hook(self, i):
        def hook(module, args, output):
            h = (output[0] if isinstance(output, tuple) else output)[0]          # [T, D]
            s = self.spans
            csum = torch.cat([h.new_zeros(1, h.size(1)), h.float().cumsum(0)])  # somme cumulative → medie veloci
            mean = (csum[s[:, 1] + 1] - csum[s[:, 0]]) / (s[:, 1] - s[:, 0] + 1)[:, None]
            self.out[i] = {"q": h[self.answer_pos], "end": h[s[:, 1]], "mean": mean.to(h.dtype)}
        return hook
```

Si lavora con una riga per forward (o righe di lunghezza simile); con più righe insieme, posizioni e intervalli vanno spostati del padding di ciascuna.

5. Hook sull'attenzione

Il modello calcola l'attenzione con funzioni veloci che **non restituiscono i pesi**, e chiederli in modo standard ("eager") costa troppa memoria con documenti lunghi. Si registra una funzione di attenzione che:

1. calcola l'attenzione normale, con la maschera del modello (causale, bidirezionale o a blocchi): stesso risultato di prima;
2. negli 8 layer pieni ricalcola **una riga** di pesi: quella della posizione di risposta verso tutti i token. Costa pochissimo.

```python
from transformers import AttentionInterface
from transformers.integrations.sdpa_attention import sdpa_attention_forward

CAPTURE = {"rows": {}, "full": {3, 7, 11, 15, 19, 23, 27, 31}, "query_pos": -1}

def sdpa_capture(module, query, key, value, attention_mask, scaling=None, **kw):
    out = sdpa_attention_forward(module, query, key, value, attention_mask, scaling=scaling, **kw)
    if module.layer_idx in CAPTURE["full"]:
        qp = CAPTURE["query_pos"]
        k = key.repeat_interleave(query.size(1) // key.size(1), dim=1)            # 16 teste di query, 4 di chiavi
        s = (query[:, :, qp:qp + 1 if qp != -1 else None] @ k.transpose(-1, -2)).float() * scaling
        if attention_mask is not None:                                            # la maschera del modello, anche
         ↪ bidirezionale
            s = s + attention_mask[:, :, qp:qp + 1 if qp != -1 else None, : s.size(-1)].float()
        CAPTURE["rows"][module.layer_idx] = s.softmax(-1)[:, :, 0]                # [B, 16, T]
    return out

AttentionInterface.register("sdpa_capture", sdpa_capture)
```

- In transformers 5.14 (quello installato) la classe di attenzione di Qwen3.5 calcola l'attenzione attraverso questa interfaccia e ha `layer_idx`; `query` e `key` arrivano già pronte (dopo le rotazioni di posizione e le normalizzazioni) e `scaling` arriva dal modello.
- Se il codice degli autori installa la propria maschera bidirezionale con un hook o con un'altra funzione di attenzione (pplx, Jeffan, d3-flash, elephant-1), `sdpa_capture` va registrata **sopra** la loro (chiamando la loro invece di `sdpa_attention_forward`), così la maschera resta la loro. Il test di §6 lo verifica.
- Per elephant-1, OneJev e Nimble la posizione di risposta è la fine del suffisso della domanda; con una sola domanda per richiesta è l'ultimo token della sequenza.

Dalla riga di pesi alla feature `att[k]`: per ogni layer e testa, la somma dei pesi sui token della frase k → 8 × 16 = 128 numeri per frase, per tutti i modelli.

6. Test

Comando: `python -m evidence.models --model knowline-4b --selftest` (poi per gli altri modelli).

Test	Come	Deve dare
Verdetto invariato	50 righe del dev del piano binario, prompt senza preambolo: P(supportato) dal forward con gli hook contro l'adapter del benchmark	differenza media < 0,005, massima < 0,02
Layer giusti	il vettore catturato dall'hook del layer i = `hidden_states[i + 1]` di transformers	identico
Attenzione giusta	su 5 prompt da ~1.000 token, confronto con la modalità "eager" **con la stessa maschera** del modello	differenza < 0,001
Maschera conservata	per i modelli bidirezionali, il verdetto con gli hook è uguale a quello senza hook	identico

Test	Come	Deve dare
Memoria	forward da 8.000 e 16.000 token con gli hook	entra nella GPU (§7)

7. GPU necessaria

Uso	Gruppo 4B	Gruppo 9B
Calcolo dei vettori da salvare	24 GB	24 GB (~19 GB di pesi) o 45 GB per documenti lunghi
Training con il modello in memoria (fase 1b)	24 GB	45 GB
Training anche del modello (fase 2)	24 GB	45 GB

Passo 2 — Documenti divisi in frasi, con le frasi di evidenza

In breve

Per addestrare e valutare la testa servono righe fatte così: un documento, un claim, l'etichetta (supportato o no), il documento **diviso in frasi** e, se il claim è supportato, **quali frasi sono l'evidenza**. Queste righe si costruiscono da dataset pubblici già presenti sulla macchina.

Si scrivono due file:

- `evidence/sentences.py`: divide un testo in frasi. È **l'unica** funzione di divisione usata ovunque (dati, training, server): se in produzione le frasi fossero tagliate in modo diverso, la testa sbaglierebbe.
- `evidence/build_pairs.py`: costruisce le righe, fonte per fonte.

Uscita: `data/evidence/pairs/<fonte>_<split>.jsonl`.

1. Una riga


```
{"id": "fever:train:75397", "source": "fever", "split": "train", "label": 1,
 "claim": "Nikolaj Coster-Waldau worked with the Fox Broadcasting Company.",
 "document": "Nikolaj Coster-Waldau\nNikolaj Coster-Waldau (born 27 July 1970) is a Danish actor ...",
 "sentences": [[0, 21], [22, 134], [135, 290]],
 "evidence": [2], "evidence_sets": [[2]],
 "evidence_quality": "gold", "doc_id": "fever:Nikolaj_Coster-Waldau"}
```


Campo	Contenuto
`label`	1 = supportato, 0 = non supportato
`sentences`	dove inizia e finisce ogni frase, in caratteri
`evidence`	numeri delle frasi di evidenza (vuoto se `label = 0`)
`evidence_sets`	se ci sono più modi alternativi di dimostrare il claim (FEVER, WiCE), uno per insieme
`evidence_quality`	da dove viene l'evidenza: `gold` (annotata da persone), `construct` (nota perché i dati sono generati, 03), `ablation` (ricavata col modello, 03), `none`
`doc_id`	identifica il documento: lo stesso documento non può stare sia in training sia in test

2. Come si divide in frasi

`sentences.split(testo)` restituisce gli intervalli delle frasi. Regole:

1. si va sempre a capo su una riga vuota, su un elemento di elenco e dopo un titolo (riga senza punto finale);
2. dentro ogni blocco si usa NLTK `sent_tokenize`;
3. frasi oltre 60 parole: si spezzano su `;` o `:` (altrimenti sulla virgola più vicina a metà);
4. frasi sotto 3 parole: si uniscono alla successiva;
5. oltre 512 frasi il documento si esclude dal training.

Le frasi di evidenza si riportano sempre sulla nostra divisione. Le fonti danno le loro frasi (FEVER, HotpotQA, SciFact, WiCE), ma servono solo a sapere *dove* nel testo sta l'evidenza. Una nostra frase è evidenza se almeno metà dei suoi caratteri, o metà della frase della fonte, cade dentro una frase di evidenza della fonte.

Controllo: per ogni fonte, almeno il 90% delle frasi di evidenza originali deve corrispondere a una sola nostra frase.

3. FEVER

Claim su Wikipedia, con le righe delle pagine che lo dimostrano.

Cosa	Dove
Claim di training	`~/.cache/huggingface/hub/datasets--fever--fever/snapshots/5f57…/v1.0/train/0000.parquet` — **una riga per ogni frase di evidenza**, da raggruppare
Claim di dev e test	`dataset/Fever/data/paper_dev.jsonl`, `paper_test.jsonl`
Pagine	`dataset/Fever/data/wiki-pages/wiki-pages/wiki-*.jsonl`: il campo `lines` contiene le frasi numerate di ogni pagina

Passi:

1. Creare una volta un indice sqlite `data/evidence/fever_pages.db` (id pagina → frasi), per trovare le pagine velocemente.
2. Raggruppare l'evidenza per claim: nel parquet, righe con lo stesso `id` e lo stesso `evidence_annotation_id` formano un insieme di evidenza.
3. Documento = le pagine citate dagli insiemi, in ordine casuale (seme fisso). Ogni pagina: titolo su una riga, poi le sue frasi.
4. Il testo di FEVER è "tokenizzato": va rimesso in forma normale. `-LRB-` → `(`, `-RRB-` → `)`, `-LSB-` → `[`, `-RSB-` → `]`, ``` `` ``` e `''` → `"`, `--` → –, niente spazio prima di `, . ; : ! ? ) 's n't`. Anche nel claim.
5. SUPPORTS → positivo con evidenza. REFUTES → negativo (si annota a parte quali frasi lo contraddicono, campo `contra`, utile in valutazione). NOT ENOUGH INFO → scartato (non ha pagine).

4. HoVer

Claim che richiedono 2–4 pagine Wikipedia: il caso tipico di evidenza sparsa.

Cosa	Dove
Claim	`dataset/HoVer/data/hover_train_release_v1.1.json`, `hover_dev_release_v1.1.json`; `supporting_facts` = coppie (titolo pagina, numero di frase)
Pagine	`dataset/HoVer/data/wiki_wo_links.db`, tabella `documents(id, text)`: il testo **non** è diviso in frasi
Frasi originali e pagine di disturbo	dataset `hotpot_qa`, configurazione `distractor` (va scaricato: in cache c'è solo il README), cercato con il campo `hpqa_id`

Il test di HoVer non ha etichette: si divide il dev in due metà (una per dev, una per test).

Passi:

1. Scaricare `hotpot_qa` (distractor) e indicizzarlo per `hpqa_id`.
2. Documento = le pagine di `supporting_facts` più le ~8 pagine di disturbo di HotpotQA per quella domanda, in ordine casuale. Così il documento è lungo e le frasi di evidenza sono distanti.
3. Dove sta l'evidenza: se la pagina è in HotpotQA, la frase numero i è la i-esima frase di HotpotQA, cercata nel testo. Se la pagina non c'è (capita con 3–4 pagine), si divide il testo con `sentences.py` e si prende la frase i, solo se il conto delle frasi è plausibile; altrimenti la riga va etichettata col modello (03) o scartata.
4. SUPPORTED → positivo; NOT_SUPPORTED → negativo.

Controllo: su 100 pagine presenti in entrambi, il testo del db deve coincidere con le frasi di HotpotQA unite (a meno degli spazi).

5. SciFact e WiCE (solo per il test, mai per il training)

Fonte	File	Cosa si tiene
SciFact	`~/.cache/huggingface/hub/datasets--allenai--scifact_entailment/snapshots/be3…/default/validation/0000.parquet` (340 righe): `abstract` (lista di frasi), `evidence` (numeri), `verdict`	documento = titolo + abstract; `SUPPORT` → positivo, `CONTRADICT` → negativo
WiCE	`~/.cache/huggingface/hub/datasets--tasksource--wice/snapshots/4905…/test.jsonl`: `evidence` (frasi), `supporting_sentences` (insiemi), `label`	`supported` → positivo; `not_supported` → negativo; `partially_supported` scartato

Servono a vedere se la testa funziona su testi diversi da quelli del training (articoli scientifici, pagine web). WiCE fa anche parte di AggreFact: per questo non entra mai nel training.

6. MiniCheck C2D e D2C

`~/.cache/huggingface/hub/datasets--lytang--C2D-and-D2C-MiniCheck/…/data/c2d-*.parquet` e `d2c-*.parquet`: documenti generati, con claim ed etichetta ma **senza** evidenza. Si dividono in frasi; ai positivi l'evidenza si aggiunge al passo 03.

7. Comandi

```
python -m evidence.build_pairs --source fever   --split train dev test
python -m evidence.build_pairs --source hover   --split train dev
python -m evidence.build_pairs --source scifact --split validation
python -m evidence.build_pairs --source wice    --split test
python -m evidence.build_pairs --source c2d d2c
```

Ogni comando scrive anche un `.stats.json`: righe per etichetta, righe scartate e perché, frasi per documento, frasi di evidenza per riga.

8. Controlli

Controllo	Atteso
Le frasi coprono tutto il documento, senza sovrapporsi	tutte le righe
Frasi di evidenza originali corrispondenti a una sola nostra frase	≥ 90% per fonte
30 righe positive per fonte lette a mano: le frasi marcate dimostrano il claim	≥ 28 su 30
10 documenti FEVER letti a mano	nessun `-LRB-`, spaziatura normale

Passo 3 — Altri dati con evidenza nota

In breve

Le righe del passo 2 hanno due limiti: l'evidenza annotata viene quasi solo da Wikipedia, e le frasi di evidenza sono spesso vicine fra loro. Per insegnare alla testa a lavorare su altri tipi di testo e a trovare frasi lontane servono tre tipi di dati in più:

Tipo	Idea	Script	`evidence_quality`
Generati con posizioni note	un LLM scrive un articolo che contiene frasi date; si sa dove sono	`evidence/gen_marked.py`	`construct`
Etichettati col modello	per documenti senza evidenza, si cerca col modello il gruppo più piccolo di frasi che basta a fargli dire "supportato"	`evidence/label_ablation.py`	`ablation`
Evidenza spostata	si prendono righe con evidenza nota e si allontanano le frasi di evidenza, o si inseriscono frasi di disturbo	`evidence/augment_sparse.py`	come la riga di partenza

Tutto si genera **solo da righe di training**. Generatore e giudice LLM sono quelli del piano binario (../fine_tuning_factcheck/02b_dataset_generazione_llm.md).

1. Documenti generati con posizioni note (C2D e CG2C)

C2D. Nella ricetta C2D del piano binario ogni claim viene scomposto in poche frasi semplici che insieme lo dimostrano. Si chiede al generatore un articolo che le contenga **marcate**:

```
Write a coherent news-style article of 15 to 40 sentences that includes each of the
following sentences word for word, each wrapped in its tag: <e1>…</e1>, <e2>…</e2>, …
Add other context sentences without tags. Do not restate the information of the
tagged sentences anywhere else in the article. Place the tagged sentences far apart.

<e1>{frase 1}</e1>
<e2>{frase 2}</e2>
```

Poi:

1. si controlla che ogni frase marcata ci sia, una volta sola, parola per parola (altrimenti si scarta);
2. si tolgono i tag ricordando dove stavano;
3. si divide il testo con `sentences.py`; evidenza = le nostre frasi che contengono i pezzi marcati;
4. un giudice LLM conferma: "queste frasi, da sole, dimostrano il claim?". Si tengono solo le righe confermate.

Dallo stesso articolo si ricavano due righe in più:

- **negativo**: si toglie una frase marcata necessaria → il claim non è più supportato;
- **claim parziale**: si riscrive il claim usando solo alcune frasi → supportato, e l'evidenza sono solo quelle frasi.

CG2C. Nella ricetta CG2C del piano binario si estrae un grafo di fatti dal documento e si costruisce il claim seguendo un percorso nel grafo. Basta salvare, per ogni fatto, la frase da cui viene: l'evidenza sono le frasi dei fatti del percorso. Danno evidenza su più frasi fuori da Wikipedia.

2. Etichette ricavate col modello (ablazione)

Per righe supportate senza evidenza (C2D e D2C come rilasciati, righe HoVer non allineabili) si usa il verdetto di **KnowLine-4B** del piano binario, con il prompt con preambolo (05). Si cerca il gruppo più piccolo di frasi che, da solo, fa dire "supportato":

```
1. si parte da un gruppo vuoto
2. si aggiunge la frase che, insieme al gruppo, dà la P(supportato) più alta
3. si ripete finché P(supportato) supera la soglia (al massimo 6 frasi; se non basta, la riga si scarta)
4. pulizia: si toglie ogni frase del gruppo senza la quale P resta sopra soglia
5. controllo: se anche il documento SENZA il gruppo resta "supportato", l'evidenza è ripetuta altrove
   → la riga si tiene con peso più basso (ablation_partial)
```

Ogni passo richiede un forward per frase candidata: si mandano tutti insieme e si riusa il calcolo della parte iniziale comune del prompt.

Filtri:

- un giudice LLM conferma le frasi trovate; atteso almeno l'80% di conferme, altrimenti si rivede la procedura;
- su 300 righe si rifà la ricerca con elephant-1: i gruppi trovati devono coincidere in buona parte (Jaccard medio ≥ 0,7). Se no, le etichette dipendono troppo da KnowLine, e per gli altri modelli si usano solo le righe su cui i due concordano.

```
gpu-run --batch --name ev-ablation-knowline -- bash -c \
  'python -m evidence.label_ablation --model knowline-4b --pairs c2d_train d2c_train hover_train_unaligned \
     --out data/evidence/pairs/ablation_knowline.jsonl > logs/ev-ablation-knowline.out 2>&1'
```

Lo script riprende da dove si era fermato se interrotto.

3. Evidenza spostata (righe sparse)

Da righe con almeno 2 frasi di evidenza (annotate o generate):

Trasformazione	Come
Agli estremi	la prima frase di evidenza va all'inizio del documento, l'ultima alla fine

Trasformazione	Come
Frasi di disturbo	si inseriscono 5–40 frasi prese da altri documenti di training della stessa fonte fra le frasi di evidenza
Lontano dal claim del preambolo	si aggiungono frasi di disturbo all'inizio finché la prima frase di evidenza è oltre 1.024 token (o oltre 4.096) dall'inizio del documento
Negativi spostati	le stesse operazioni su righe non supportate

Controlli:

- una frase che comincia riferendosi alla precedente ("He", "It", "This", "The company"…) non si sposta da sola: si sposta insieme alla precedente, o si scarta la riga;
- il giudice conferma che l'etichetta del documento modificato non è cambiata.

```
python -m evidence.augment_sparse --pairs fever_train hover_train construct --n 10000 --seed 0
```

4. Quante righe

Tipo	Prima prova	Versione completa
C2D con posizioni	2k	10k
CG2C con posizioni	—	5–10k
Etichettate col modello	2k	10–20k
Evidenza spostata (positivi)	4k	10k
Negativi spostati	1k	3k

5. Controlli

Controllo	Atteso
30 righe per tipo lette a mano: le frasi di evidenza dimostrano il claim e non ne manca nessuna necessaria	≥ 27 su 30
Conferme del giudice sull'ablazione	≥ 80%
Accordo KnowLine–elephant-1 sull'ablazione	Jaccard ≥ 0,7
Righe spostate con riferimenti rotti (30 lette)	0

Passo 4 — Split e mix di training

In breve

Due cose:

1. **Split**: si decide una volta per tutte quali righe servono per scegliere (dev) e quali per la misura finale (test), e non si cambiano più. Nessun documento può stare sia in training sia in dev/test.
2. **Mix**: si costruisce l'insieme di training della testa, con le giuste proporzioni di casi facili, difficili e negativi.

Script: `evidence/make_splits.py`, `evidence/make_mix.py`. Gli split sono gli stessi per tutti i modelli.

1. Regole degli split

- Si parte dagli split del piano binario (../fine_tuning_factcheck/00b_split_e_protocollo.md): una riga che lì è in dev o test resta in dev o test.

- Si divide per documento (`doc_id`), non per riga.
- FEVER: train → training, `paper_dev` → dev, `paper_test` → test. HoVer: il dev ufficiale si divide a metà per documento (dev / test).
- I dati generati e spostati nascono solo da righe di training, con frasi di disturbo di training.

2. I set di valutazione

Set	Che cosa contiene	Righe	A che cosa serve
`dev_evid_select`	positivi con evidenza nota del dev (FEVER, HoVer, C2D con posizioni) + versioni spostate	3.000	tutte le scelte: configurazione, soglie, esperimenti
`dev_evid_confirm`	come sopra, altre righe	2.000	conferma delle scelte
`dev_evid_neg`	negativi del dev con documenti lunghi (FEVER REFUTES, HoVer NOT_SUPPORTED)	1.500	controllare che la testa non dia punteggi alti a frasi su claim non supportati
`test_evid_id`	positivi del test di FEVER e HoVer, 10% di C2D	3.000	misura finale, testi simili al training
`test_evid_ood`	SciFact, WiCE	tutte	misura finale, testi diversi
`test_evid_sparse`	da `test_evid_id`, con evidenza spostata lontano	1.500	misura finale, evidenza lontana

Per ogni set si salvano anche:

- quante frasi di evidenza ha ogni riga;
- quanto sono lontane fra loro (vicine / 2–9 / 10–19 / 20 o più frasi);
- quanto è lontana l'ultima frase di evidenza dal claim del preambolo, in token (meno di 1.024 / 1.024–4.096 / oltre 4.096), con il tokenizer di Qwen3.5 (lo stesso per tutti i modelli; le distanze cambiano solo di poco per le differenze di prompt).

```
python -m evidence.make_splits --binary-splits <manifest del piano binario> --out data/evidence/splits/
```

3. Il mix `ev_head` (training della sola testa)

Ogni riga ha un tipo, che dice come la loss la usa (08 §3):

- `supervised`: positivo con evidenza nota → le frasi di evidenza devono superare la soglia, le altre no;
- `mil_neg`: negativo → **nessuna** frase deve superare la soglia;
- `mil_pos`: positivo senza evidenza nota → almeno una frase deve superare la soglia (solo nell'esperimento H-MIL).

Famiglia	Righe	Tipo	Peso della riga	Peso delle frasi non di evidenza
FEVER supportati	6.000	`supervised`	1,0	0,3
HoVer supportati	5.000	`supervised`	1,0	0,5

Famiglia	Righe	Tipo	Peso della riga	Peso delle frasi non di evidenza
C2D / CG2C con posizioni	2.000	`supervised`	1,0	1,0
Etichettati col modello	3.000	`supervised`	0,5 (ripetuti altrove: 0,3)	0,5
Evidenza spostata	4.000	come l'origine	come l'origine	come l'origine
Negativi con documenti lunghi	6.000	`mil_neg`	1,0	—
Positivi senza evidenza (solo H-MIL)	3.000	`mil_pos`	1,0	—

Perché il peso delle frasi non di evidenza è basso per FEVER (0,3): in FEVER gli annotatori hanno segnato un gruppo di frasi sufficiente, non tutte. Una frase non segnata può essere evidenza valida, quindi la si penalizza poco.

Lo script controlla questi vincoli e si ferma se non sono rispettati:

Vincolo	Valore
Positivi con 2 o più frasi di evidenza	≥ 40%
Positivi con evidenza distante 10 o più frasi	≥ 25%
Positivi con evidenza oltre 1.024 token dal preambolo	≥ 15%
Positivi da testi non Wikipedia	≥ 30%
Documenti con più di 50 frasi	≥ 15%
Negativi	20–25%
Negativi con una frase molto simile al claim (per parole)	≥ 50% dei negativi
Documenti in comune con dev o test	0

4. Il mix `ev_joint` (training anche del modello, fase 2)

Il mix di training del piano binario (../fine_tuning_factcheck/02e_mix_finale.md), in cui le righe che hanno evidenza la portano con sé, più le righe di `ev_head` (fino al 20–30% del totale). Tutte le righe servono al verdetto; il termine di evidenza si applica solo dove c'è un tipo.

```
python -m evidence.make_mix --recipe recipes/ev_head.json  --out data/evidence/mixes/ev_head.jsonl
python -m evidence.make_mix --recipe recipes/ev_joint.json --out data/evidence/mixes/ev_joint.jsonl
```

La ricetta (`recipes/*.json`) contiene famiglie, numeri, pesi e seme: lo stesso mix si ricostruisce identico.

Passo 5 — Il prompt con il preambolo

In breve

Si aggiunge una riga con il claim **prima** del documento, lasciando il resto del prompt identico a quello del piano binario. Poi si controlla che il verdetto del piano binario funzioni ancora con questa riga in più. Se sì, si riusa; se no, si riaddestra con la stessa ricetta del piano binario, includendo la riga.

Perché serve: nei modelli causali (KnowLine, OneJev, Nimble) ogni token vede solo quello che lo precede. Senza preambolo, quando il modello legge il documento non sa ancora che cosa deve verificare, e i vettori delle frasi non "parlano" del claim. Nei modelli bidirezionali le frasi vedono

il claim negli 8 layer pieni, ma non nei 24 layer DeltaNet, che restano causali: il preambolo serve meno, e l'esperimento H4 (11) misura quanto.

1. Esempio (KnowLine-4B, schema)

Prompt del piano binario (schema: il testo esatto lo produce il front end degli autori, `knowline_layout.py`):

```
[turno utente]       document: The meeting was moved to Tuesday at 10:00. It will be held in Rome.
                     claim: The meeting is on Tuesday.
[turno utente]       <istruzione> Question: <domanda del piano binario> Options: A. Yes: … B. No: …
[turno assistente]   Answer:
```

Prompt con il preambolo:

```
[turno utente]       Claim under review: The meeting is on Tuesday.

                     document: The meeting was moved to Tuesday at 10:00. It will be held in Rome.
                     claim: The meeting is on Tuesday.
[turno utente]       <istruzione> Question: <domanda del piano binario> Options: A. Yes: … B. No: …
[turno assistente]   Answer:
```

Unica differenza: la riga `Claim under review: …` e una riga vuota, all'inizio del turno con lo stato. Il verdetto si legge sempre nello stesso punto. Nelle richieste che KnowLine scrive due volte (fino a 3.000 token), il preambolo compare in entrambe le copie, perché fa parte dello stato.

2. Dove va il preambolo negli altri modelli

Regola unica: **nello stesso turno dello stato, subito prima dello stato**, con lo stesso escaping usato per lo stato. Si aggiunge come opzione `preamble=` del renderer del piano binario, che chiama il codice degli autori (../fine_tuning_factcheck/03_rendering_formato.md §2).

Modello	Lo stato nel prompt	Dove va il preambolo
KnowLine-4B	testo indentato in un turno utente	prima dello stato (esempio sopra)
elephant-1	`<state>` JSON indentato `</state>` nel prefisso comune	prima di `<state>`: resta nel prefisso, quindi è visto da tutte le domande
StartLux 9B/4B	`Evidence:` + JSON compatto	prima di `Evidence:`
d3-flash, Jeffan-4B	`State:` + JSON	prima di `State:`
OneJev1.6-9B	blocco dello stato nel prefisso comune	prima del blocco dello stato
Nimble-9B-v3	turno utente JSON con contesto e schema	come campo testuale aggiuntivo **prima** del contesto, se il motore accetta un prefisso nel turno; altrimenti si salta (Nimble esce dagli esperimenti sul preambolo)
pplx-decider-flash	sequenza unica con lo stato e le domande	prima dello stato
Clef-Flash	`STATE:` + JSON compatto	prima di `STATE:`

Test: con `preamble=None`, `input_ids` identici token per token a quelli del piano binario; con il preambolo, i token dopo il preambolo identici a quelli del prompt originale (a meno di un token al confine).

3. Scegliere il testo del preambolo

Due candidati:

- **P-A**: `Claim under review: <claim>`

- **P-B**: `The following document will be checked against this claim: <claim>`

Sul dev del piano binario (`dev_select`), per ciascun modello, si misurano BAcc, AUC ed ECE (calibrazione) in sei condizioni:

Condizione	Modello del verdetto	Prompt
B-1D	di partenza	senza preambolo
B-SAND-A, B-SAND-B	di partenza	P-A, P-B
B-FT	addestrato nel piano binario	senza preambolo
B-FT-SAND-A, B-FT-SAND-B	addestrato nel piano binario	P-A, P-B

Si sceglie il testo con cui B-FT-SAND è più vicino a B-FT in AUC; a parità (entro 0,2 punti), il più corto. Lo stesso testo per tutti i modelli.

4. Riusare il verdetto o riaddestrarlo

Per ciascun modello:

Risultato su `dev_select`	Cosa fare
B-FT-SAND entro 0,3 punti di BAcc e AUC da B-FT, ECE entro 0,01	si usa il verdetto del piano binario così com'è (si ricalcola solo la soglia, se si sposta)
peggio di così	si riaddestra il verdetto con la ricetta del piano binario e il preambolo nel prompt

5. Dove si scrive la scelta

In `question.json` del piano binario si aggiungono due campi, letti da tutti gli script:

```
"preamble": {"template": "Claim under review: {claim}", "chosen_on": "dev_select"},
"verdict":  {"knowline-4b": "binary_plan", "elephant-1": "retrained_with_preamble"}
```

(Valori di esempio.)

Passo 6 — Dalle frasi ai token, e salvataggio dei vettori

In breve

1. **Rendering** (`evidence/render.py`): per ogni riga si costruisce il prompt con il preambolo e si trova, per ogni frase, **quali token del prompt le appartengono** (primo e ultimo). È il dato da cui dipende tutto: se le posizioni sono sbagliate anche di un token, la testa legge i vettori sbagliati.
2. **Estrazione** (`evidence/extract.py`): un forward per riga con gli hook del passo 01; si salvano su disco i vettori di ogni frase. Il modello è congelato, quindi questi vettori non cambiano mai: si calcolano una volta e la testa si addestra su di essi in pochi minuti.

1. L'esempio pronto per il modello

```
{"id": "hover:train:...", "model": "knowline-4b",
 "input_ids": [1, 2, "..."], "answer_pos": 1287,
 "sent_tok_spans": [[198, 212], [213, 248], "…", [1201, 1240]],
 "sent_char_spans": [[0, 112], [113, 260], "…"],
 "label": 1, "evidence_target": [1, 0, 0, "…", 1],
 "evidence_term": "supervised", "evidence_weight": 1.0, "omega": 0.3}
```

- `sent_tok_spans[k]`: primo e ultimo token della frase k nel prompt.
- `sent_char_spans[k]`: inizio e fine della frase k nel documento originale (servono al server per evidenziare).

- `answer_pos`: l'ultimo token del prompt.
- `evidence_target`: 1 per le frasi di evidenza, 0 per le altre.

2. Trovare i token di ogni frase

Il problema: in quasi tutti i modelli il documento è dentro un JSON (elephant-1, StartLux, d3-flash, Jeffan, Nimble, pplx, Clef), quindi nel prompt non è scritto come nell'originale. Virgolette e a capo diventano sequenze più lunghe: `"` → `\"`, a capo → `\n`, `<` → `<`. Le posizioni delle frasi nel testo originale non coincidono con quelle nel prompt.

Procedura:

1. costruire il prompt in tre pezzi: prima del documento, documento come appare nel prompt, dopo il documento;
2. mentre si trasforma il documento, registrare dove finisce ogni carattere originale (funzione sotto);
3. tokenizzare il prompt chiedendo al tokenizer la posizione in caratteri di ogni token (`return_offsets_mapping`);
4. per ogni frase, prendere il primo e l'ultimo token che contengono suoi caratteri.

```
ESC = {'"': '\\"', "\\": "\\\\", "\n": "\\n", "\r": "\\r", "\t": "\\t", "\b": "\\b", "\f": "\\f"}   # come
↪ json.dumps

def escape_with_map(text, escape_lt=False):
    """Il testo come appare in una stringa JSON e, per ogni carattere originale (più la fine), la sua posizione
    ↪ nel testo trasformato."""
    out, pos, at = [], [], 0
    for ch in text:
        pos.append(at)
        if ch in ESC:
            piece = ESC[ch]
        elif escape_lt and ch == "<":
            piece = "\\u003c"
        elif ord(ch) < 0x20:
            piece = f"\\u{ord(ch):04x}"
        else:
            piece = ch
        out.append(piece)
        at += len(piece)
    pos.append(at)
    return "".join(out), pos
```

Verificata contro `json.dumps(…, ensure_ascii=False)` su 20.000 stringhe con caratteri difficili: identica. `escape_lt=True` serve solo se un modello scrive `<` come `\u003c` (lo si vede confrontando con il suo renderer).

Casi particolari:

- un token a cavallo di due frasi (es. `."`) va alla frase che contiene il suo ultimo carattere non spazio;
- se il codice degli autori tokenizza separatamente prefisso e domanda (elephant-1, OneJev, Nimble), le posizioni si calcolano sui pezzi e si sommano;
- KnowLine scrive lo stato come testo indentato, non JSON: il documento compare così com'è (a meno dell'indentazione delle righe), ed è ripetuto nelle richieste lette due volte: si usano gli intervalli della seconda copia.

3. Documenti troppo lunghi

Uso	Lunghezza massima del prompt
Estrazione	modelli 4B 16.384 token; modelli 9B 8.192 (16.384 se la memoria basta)
Training con il modello in memoria	8.192
Server	32.768

Righe di training oltre il limite: per i positivi si tolgono frasi **non** di evidenza, a partire dalle più lontane dall'evidenza (le etichette restano valide); per i negativi si taglia la fine. Le righe di dev e test non si tagliano: se sono troppo lunghe si escludono e si contano.

4. Che cosa si salva

Per ogni riga, dal forward:

Nome	Forma	Che cos'è
`q`	4 layer × d	vettore della posizione di risposta
`h_end`	n frasi × 4 layer × d	vettore dell'ultimo token di ogni frase
`h_mean`	n frasi × 4 layer × d	media dei vettori dei token di ogni frase
`att`	n frasi × 8 layer × teste	quanta attenzione la posizione di risposta dà ai token di ogni frase
`n_tok`	n frasi	quanti token ha ogni frase
`answer_logits`	2	punteggi di "sì" e "no", per ricalcolare P(supportato)

Spazio per 26.000 righe con ~60 frasi: ~60 GB per un modello 4B, ~100 GB per un 9B. Se lo spazio non basta, per i modelli successivi si salvano solo i layer che su KnowLine sono risultati utili (esperimento H2a).

Formato: `data/evidence/features/<modello>/<set>/shard-00000.safetensors` (1.000 righe per file) più un indice `index.jsonl`. Ogni cartella ha un `meta.json` con modello, pesi, preambolo, layer e versione del codice di divisione in frasi e di rendering: il training rifiuta vettori calcolati con impostazioni diverse.

5. Comando

```
gpu-run --batch --name ev-extract-knowline -- bash -c \
  'python -m evidence.extract --model knowline-4b \
     --rows data/evidence/mixes/ev_head.jsonl data/evidence/splits/dev_evid_select
↪  data/evidence/splits/dev_evid_confirm data/evidence/splits/dev_evid_neg \
     --out data/evidence/features/knowline-4b > logs/ev-extract-knowline.out 2>&1'
```

Se si interrompe, riparte dai file già completati.

6. Test

Test	Deve dare
I token di ogni frase, ritrasformati in testo, sono la frase (200 righe per modello)	200 su 200
Documenti con ", \, a capo, tab, <, emoji, cinese, arabo	posizioni giuste per ogni modello
Senza preambolo, il prompt è identico a quello del piano binario	identico
P(supportato) dai punteggi salvati = P del verdetto scelto al passo 5	differenza < 0,005
Stessa riga estratta due volte	vettori identici

Passo 7 — Metodi semplici da battere

In breve

Prima di addestrare la testa si misura quanto ottengono metodi semplici, sullo stesso set (`dev_evid_select`) e con le stesse metriche (10). Servono a due cose: sapere se la testa vale la pena (deve batterli con margine) e capire quanto è difficile il problema sui nostri dati (soprattutto con l'evidenza lontana).

Script: `evidence/baselines.py`.

1. I metodi

Nome	Come dà un punteggio a ogni frase	Costo
E-RAND	a caso	nullo; è il pavimento
E-LEX	quante parole ha in comune col claim (BM25)	nullo; è il riferimento principale da battere
E-EMB	somiglianza di significato fra frase e claim, con un piccolo modello di embedding (es. `BAAI/bge-small-en-v1.5`)	un modello in più
E-ATT	quanta attenzione il decision model dà alla frase dalla posizione di risposta (la feature `att` già salvata al passo 6)	nullo: legge i dati salvati
E-ABL	gruppo di frasi trovato con l'ablazione del passo 03 §2, col modello stesso	molti forward per riga: solo come riferimento

E-LEX ed E-EMB non possono trovare una frase ponte che non ha parole in comune col claim (come la frase 1 dell'esempio Nancy Howell): sull'evidenza lontana ci si aspetta che crollino.

E-ABL è il riferimento "di qualità": dice quanto ci si avvicina, con un solo forward, a un metodo che ne usa centinaia.

2. Come si calcola E-ATT

L'attenzione è misurata in 8 layer e 16–32 teste: non tutte le combinazioni sono utili.

1. Si divide `dev_evid_select` in due metà, A e B.
2. Su A si prova ogni combinazione di layer e testa, con il punteggio diviso o no per la lunghezza della frase, e si tengono le 8 migliori.
3. Il punteggio finale è la media delle 8; si misura su B.

Si calcola sia con il preambolo sia senza (per il secondo serve un'estrazione col prompt senza preambolo, solo su `dev_evid_select`): dice se il preambolo aiuta il modello a guardare le frasi giuste.

3. Quali frasi si mostrano

I metodi semplici non hanno una soglia propria. Per ciascuno si prova:

- le **k migliori**, con k = numero medio di frasi di evidenza;
- una **soglia** sul punteggio, scelta su A (da 1 a 5 frasi).

Si riporta la migliore delle due, misurata su B.

4. Comandi

```
python -m evidence.baselines --method rand lex emb --block dev_evid_select
python -m evidence.baselines --method att --model knowline-4b --block dev_evid_select
gpu-run --batch --name ev-eabl-knowline -- bash -c \
  'python -m evidence.baselines --method abl --model knowline-4b --block dev_evid_select --per-source 300 >
  ↪  logs/ev-eabl-knowline.out 2>&1'
```

E-ABL: 300 righe per fonte e per modello.

5. Tabella da compilare (una per modello)

Metodo	F1 FEVER	F1 HoVer	Insieme completo HoVer	F1 evidenza a ≥ 10 frasi	F1 evidenza oltre 1.024 token
E-RAND					
E-LEX					
E-EMB					
E-ATT con preambolo					
E-ATT senza preambolo					
E-ABL					

Risultati in `data/evidence/runs/baselines.json`.

Passo 8 — La testa e la loss

In breve

- La **testa** (`evidence/head.py`) riceve i vettori salvati al passo 6 e restituisce un punteggio per ogni frase (`s_k`) e un punteggio-soglia per la riga (`s_0`).
- La **loss** (`evidence/losses.py`) dice alla testa quanto sta sbagliando, così il training la corregge.

Ogni parte della testa si può accendere o spegnere con un'opzione: tutti gli esperimenti (11) usano lo stesso codice con opzioni diverse.

1. La testa, passo per passo

Per una riga con n frasi:

1. **Combina i layer.** Per ogni frase ci sono vettori da 4 layer. La testa li somma con 4 pesi che impara (normalizzati in modo che sommino a 1).
2. **Costruisce un vettore per frase** da 256 numeri:
   - vettore dell'ultimo token e media dei token della frase, affiancati e moltiplicati per una matrice;
   - più l'attenzione ricevuta dalla frase (8 layer × teste, più la lunghezza della frase), moltiplicata per un'altra matrice;
   - più un vettore che indica la posizione della frase nel documento (frase 0, 1, 2, …).
3. **Costruisce il vettore `x_0`** da `q` (la posizione di risposta), allo stesso modo.
4. **Fa comunicare tutto**: `[x_0, frase 0, …, frase n−1]` passano in 2 layer di transformer in cui ognuno vede tutti gli altri. Dopo, il vettore della frase 1 "sa" anche della frase 40.
5. **Dà i punteggi**: uno stesso strato finale trasforma ogni vettore in un numero. Il numero di `x_0` è la soglia `s_0`; quelli delle frasi sono `s_k`.

Facoltativo (esperimento H-PAIR): un punteggio in più per ogni **coppia** di frasi, che premia una frase se c'è un'altra frase con cui "fa coppia". Pensato per l'evidenza su più frasi.

```python
import torch
import torch.nn as nn

class EvidenceHead(nn.Module):
    def __init__(self, d_model, n_mix, n_att, d=256, layers=2, heads=4, max_sent=512,
                 use_mix=True, use_mean=True, use_att=True, use_pos=True, use_pair=False, pair_rank=64):
        super().__init__()
        self.use_mix, self.use_mean, self.use_att, self.use_pos, self.use_pair = use_mix, use_mean, use_att,
        ↪  use_pos, use_pair
        self.mix = nn.Parameter(torch.zeros(n_mix))                    # pesi dei layer (punto 1)
        self.ln = nn.LayerNorm(d_model)                                 # i layer hanno scale diverse
        self.proj_h = nn.Linear(d_model * (2 if use_mean else 1), d)    # [ultimo token ; media] → 256
        self.proj_q = nn.Linear(d_model, d)                             # q → 256
        self.proj_a = nn.Linear(n_att + 1, d)                          # attenzione + log(numero di token) → 256
        self.pos = nn.Embedding(max_sent, d)                            # posizione della frase
        self.kind = nn.Embedding(2, d)                                  # 0 = x_0, 1 = frase
        enc = nn.TransformerEncoderLayer(d, heads, 4 * d, dropout=0.1, batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(enc, layers)               # tutti vedono tutti (punto 4)
        self.out = nn.Linear(d, 1)                                      # punteggio (punto 5)
        self.pair = nn.ModuleList([nn.Linear(d, pair_rank, bias=False), nn.Linear(d, pair_rank, bias=False)])
        self.pair_tau = nn.Parameter(torch.tensor(0.1))

    def mixed(self, x):                                                 # x: [..., n_mix, D]
        if not self.use_mix:
            return self.ln(x[..., -1, :])                               # solo l'ultimo layer
        return self.ln((self.mix.softmax(0)[:, None] * x).sum(-2))

    def forward(self, q, h_end, h_mean, att, n_tok, sent_mask):
        """q: [B, L, D]; h_end, h_mean: [B, N, L, D]; att: [B, N, n_att]; n_tok, sent_mask: [B, N] (1 = frase
        ↪  vera).
        Restituisce s: [B, N], s0: [B], S: [B, N, N] o None."""
        h = self.mixed(h_end)
        if self.use_mean:
            h = torch.cat([h, self.mixed(h_mean)], -1)
        x_h = self.proj_h(h) + self.kind.weight[1]
        if self.use_pos:
            x_h = x_h + self.pos(torch.arange(h.size(1), device=h.device))
        if self.use_att:
            x_h = x_h + self.proj_a(torch.cat([att, n_tok.clamp_min(1).float().log()[..., None]], -1))
        x_0 = (self.proj_q(self.mixed(q)) + self.kind.weight[0])[:, None]
        pad = torch.cat([torch.zeros_like(sent_mask[:, :1]), 1 - sent_mask], 1).bool()
        X = self.encoder(torch.cat([x_0, x_h], 1), src_key_padding_mask=pad)
        o = self.out(X)[..., 0]
        s, s0, S = o[:, 1:], o[:, 0], None
        if self.use_pair:
            Xs = X[:, 1:]
            S = self.pair[0](Xs) @ self.pair[1](Xs).transpose(1, 2)    # punteggio di ogni coppia
            invalid = ~sent_mask.bool()[:, None, :] | torch.eye(S.size(1), dtype=torch.bool, device=S.device)
            support = (S + s.detach()[:, None, :]).masked_fill(invalid, float("-inf")).amax(-1)
            s = s + self.pair_tau * support.nan_to_num(neginf=0.0)      # nessun partner: nessun contributo
        return s, s0, S
```

Circa 2–4 milioni di parametri. Righe con meno frasi si riempiono fino alla più lunga del batch (`sent_mask = 0` sulle frasi di riempimento).

Configurazione di partenza (H0): solo l'ultimo layer, solo l'ultimo token, niente attenzione, niente coppie; loss del §2; soglia fissa. Da qui gli esperimenti aggiungono un pezzo alla volta.

2. Loss di partenza (H0): per frase, con soglia fissa

Ogni frase è trattata come un sì/no indipendente ("è evidenza?"), con due correzioni:

- le frasi di evidenza sono poche (2 su 50, per esempio): senza correzione la testa imparerebbe a dire sempre "no". Si pesano di più (`alpha`, ~10);
- un termine di **ordinamento**: ogni frase di evidenza deve avere un punteggio più alto, di un margine, della migliore frase non di evidenza.

In uso si mostrano le frasi con probabilità sopra una soglia `τ_e`, scelta sul dev.

```python
import torch.nn.functional as F
```

```python
def ref_loss(s, gold, sent_mask, omega, alpha=10.0, gamma=0.5, margin=1.0):
    """s, gold, sent_mask, omega: [B, N] (omega = peso delle frasi non di evidenza). Restituisce [B]."""
    bce = F.binary_cross_entropy_with_logits(s, gold.float(), reduction="none")
    w = torch.where(gold > 0, torch.full_like(s, alpha), omega) * sent_mask
    l_bce = (bce * w).sum(1) / sent_mask.sum(1)
    best_neg = s.masked_fill((gold > 0) | (sent_mask == 0), float("-inf")).amax(1, keepdim=True)
    rank = F.softplus(margin - s + best_neg).nan_to_num(posinf=0.0) * (gold > 0)
    l_rank = rank.sum(1) / (gold > 0).sum(1).clamp(min=1)
    return l_bce + gamma * l_rank
```

3. Loss con soglia per riga (quella da provare in H-LOSS)

Qui la regola è: **una frase è evidenza se il suo punteggio supera quello della soglia `s_0` della stessa riga.** La loss spinge:

- le frasi di evidenza **sopra** `s_0`, le altre **sotto** (righe `supervised`);
- **tutte** le frasi sotto `s_0` nei claim non supportati (righe `mil_neg`): così la testa impara che una frase che somiglia al claim ma non lo dimostra non va mostrata;
- almeno una frase sopra `s_0`, ma poche, nei positivi senza evidenza nota (righe `mil_pos`, solo nell'esperimento H-MIL).

Vantaggi: non c'è una soglia fissa da tarare, e il numero di frasi mostrate si adatta al caso (con una catena di 3 frasi la soglia può scendere per farle passare tutte).

In formula, con G = frasi di evidenza e ω = peso delle frasi non di evidenza:

```
supervised:  log(1 + Σ_{i ∉ G} ω_i · e^{s_i − s_0})  +  log(1 + Σ_{j ∈ G} e^{s_0 − s_j})
mil_neg:     μ_neg · log(1 + Σ_i e^{s_i − s_0})                                  μ_neg = 0,3
mil_pos:     μ_pos · [ log(1 + e^{s_0 − LSE_j s_j}) + ρ · media_i σ(s_i − s_0) ]   μ_pos = 0,3, ρ = 0,1
```

Il primo termine cresce se qualche frase non di evidenza sta sopra la soglia, il secondo se qualche frase di evidenza sta sotto.

```python
def zlpr_loss(s, s0, gold, sent_mask, omega, term, mu_neg=0.3, mu_pos=0.3, rho=0.1):
    """term: [B] con 0 = supervised, 1 = mil_neg, 2 = mil_pos, 3 = nessun termine. Restituisce [B]."""
    ninf = torch.finfo(s.dtype).min
    real = sent_mask.bool()
    d = s - s0[:, None]
    is_neg = (gold == 0) & real
    l_neg = F.softplus(torch.logsumexp((d + omega.clamp_min(1e-6).log()).masked_fill(~is_neg, ninf), 1))
    l_pos = F.softplus(torch.logsumexp((-d).masked_fill(~(gold > 0), ninf), 1)) * (gold.sum(1) > 0)
    supervised = l_neg + l_pos
    lse = torch.logsumexp(d.masked_fill(~real, ninf), 1)
    mil_neg = mu_neg * F.softplus(lse)
    spars = (torch.sigmoid(d) * sent_mask).sum(1) / sent_mask.sum(1)
    mil_pos = mu_pos * (F.softplus(-lse) + rho * spars)
    out = torch.stack([supervised, mil_neg, mil_pos, torch.zeros_like(lse)], 1)
    return out.gather(1, term[:, None]).squeeze(1)
```

La loss di ogni riga si moltiplica per il suo peso (`evidence_weight`) e si fa la media sul batch.

Esempio numerico (anche come test automatico)

40 frasi, evidenza = frasi 1 e 40, soglia `s_0 = 0`, ω = 1:

	frase 1	frase 40	una frase che somiglia al claim	altre 37	Loss
prima	2,0	−0,5 (sotto soglia: verrebbe persa)	0,5 (sopra soglia: verrebbe mostrata)	−3	**2,526**
dopo	2,0	1,5	−1,0	−3	**1,473**

La loss scende quando la frase 40 sale sopra la soglia e la frase che somiglia scende sotto.

Test per la loss di partenza: 40 frasi, evidenza {1, 40}, α = 10, tutte con probabilità 0,02 → parte per frase = **1,975**.

Il codice di questo file è stato eseguito su dati finti: forme giuste, gradienti, caso con una sola frase, e i valori sopra tornano.

4. Termine aggiuntivo per le coppie (solo nell'esperimento H-PAIR)

Coppie (H-PAIR). Nelle righe con 2 o più frasi di evidenza, il punteggio di coppia deve essere alto per le coppie di frasi di evidenza e basso fra una frase di evidenza e le 5 frasi non di evidenza più "simili". Peso 0,2.

```
def pair_loss(S, gold, sent_mask, n_hard=5):
    g = gold.bool() & sent_mask.bool()
    pos_pairs = g[:, :, None] & g[:, None, :] & ~torch.eye(S.size(1), dtype=torch.bool, device=S.device)
    l_pos = F.binary_cross_entropy_with_logits(S[pos_pairs], torch.ones_like(S[pos_pairs]))
    neg_cand = S.masked_fill(~(g[:, :, None] & (~g & sent_mask.bool())[:, None, :]), float("-inf"))
    hard = neg_cand.topk(min(n_hard, S.size(1)), -1).values
    hard = hard[torch.isfinite(hard)]
    l_neg = F.binary_cross_entropy_with_logits(hard, torch.zeros_like(hard))
    return l_pos + l_neg
```

5. Test

`tests/test_evidence_losses.py`: i valori del §3 (tolleranza 0,001); con tutte le frasi molto sotto soglia `mil_neg` ≈ 0; una riga senza termine dà loss e gradiente zero. `tests/test_evidence_head.py`: forme, frasi di riempimento ignorate, riga con una sola frase, con e senza coppie.

Passo 9 — Training

In breve

Si addestra la testa in tre modi, dal più semplice al più costoso. Si passa al successivo solo se il precedente non basta.

Fase	Il modello	Su cosa si addestra la testa	Quando
1a	congelato, non in memoria	sui vettori salvati al passo 6	quasi sempre: ogni prova dura minuti
1b	congelato, in memoria	sui vettori ricalcolati a ogni passo	solo per l'esperimento che ha bisogno dei vettori di tutti i token (H2d)
2	addestrato anche lui, con una LoRA	come 1a	solo se, alla fine degli esperimenti, gli obiettivi non sono raggiunti

Nelle fasi 1a e 1b il verdetto **non può cambiare**: il modello non viene toccato.

Ogni prova scrive in `data/evidence/runs/<modello>/<id>/`: `config.json` (tutte le opzioni), `head.safetensors` (pesi della testa), `log.jsonl` (andamento), `dev_metrics.json`.

1. Fase 1a

Impostazione	Valore
Ottimizzatore	AdamW, learning rate 1e-3 con riscaldamento iniziale (5%) e discesa a coseno, weight decay 0,01
Epoche	fino a 10; ci si ferma se per 2 epoche `dev_evid_select` non migliora
Batch	64 righe
Dropout	0,1; in più, per ogni frase, la feature di attenzione si azzera con probabilità 0,1 (per non dipendere da una sola testa)
Checkpoint tenuto	quello con la media più alta di F1 e insieme completo su `dev_evid_select`

```
python -m evidence.train_head --phase 1a --model knowline-4b --features data/evidence/features/knowline-4b \
    --mix ev_head --config configs/h0.json --run h0
```

Ogni esperimento è un file in `configs/` con le opzioni della testa e della loss.

Prima della prima prova: la testa deve riuscire a imparare a memoria 64 righe (F1 ≈ 1 su quelle righe). Se non ci riesce, c'è un errore nei dati o nel codice.

2. Quali frasi si mostrano

Si decide dopo il training, su `dev_evid_select`, guardando solo le righe che il verdetto giudica supportate:

Loss usata	Regola	Cosa si sceglie
loss di partenza	probabilità della frase > `τ_e`	il valore di `τ_e` che dà l'F1 migliore
loss con soglia per riga	`s_k > s_0 + δ`	δ fra −1 e 1; si tiene δ diverso da 0 solo se guadagna almeno 1 punto di F1

Sempre almeno 1 frase (la migliore) e al massimo 5. Si conferma su `dev_evid_confirm`: l'F1 non deve scendere di più di 1 punto.

3. Che cosa guardare durante il training

Cosa	Su	Segnale di problema	Cosa fare
F1 e insieme completo	`dev_evid_select`	non supera E-LEX dopo 2 epoche	controllare le posizioni dei token (06 §6) prima di cambiare la testa
F1 per distanza fra frasi di evidenza	righe spostate	crolla oltre 10 frasi	più righe spostate; esperimenti H3c, H-PAIR
F1 per distanza dal preambolo	`dev_evid_select`	crolla oltre 1.024 o oltre 4.096 token	più righe lontane; attenzione come feature (H2b)
Frasi mostrate per riga	`dev_evid_select`	molto più o molto meno delle frasi di evidenza	rivedere δ o il peso dei negativi
Negativi con frasi sopra soglia	`dev_evid_neg`	oltre il 30%	peso dei negativi più alto

Cosa	Su	Segnale di problema	Cosa fare
Positivi incerti (P appena sopra la soglia del verdetto)	`dev_evid_select`	la testa non mostra quasi nulla	peso dei negativi più basso
Pesi dei 4 layer	testa	tutto su un layer	basta quel layer (meno spazio, meno latenza)

4. Fase 1b

Il modello è caricato in memoria e congelato; a ogni passo si rifà il forward per avere i vettori. Serve per un solo esperimento:

- **H2d**: invece della media fissa dei token di una frase, una media pesata in cui la testa decide quali token contano (per esempio il nome proprio dentro una frase lunga). Richiede i vettori di tutti i token, che non si salvano su disco.

Impostazione	Gruppo 4B	Gruppo 9B
GPU	24 GB	45 GB
Righe per passo	4–8 (accumulando fino a 32)	1–2 (accumulando fino a 32)
Lunghezza massima	8.192 token	8.192 token
Learning rate	5e-4	5e-4
Partenza	la testa migliore della fase 1a	idem

5. Fase 2: training anche del modello

Solo se gli obiettivi non sono raggiunti dopo gli esperimenti della fase 1.

Impostazione	Valore
Partenza	LoRA del verdetto del piano binario + testa migliore della fase 1
Learning rate	LoRA 5e-6 – 1e-5 (basso: il verdetto è già addestrato); testa 2e-4
Loss	loss del verdetto del piano binario + 0,3 × loss di evidenza
Dati	`ev_joint`
Epoche	1

Il verdetto ora può cambiare, quindi:

- prima del primo passo si controlla che il verdetto sia identico a quello di partenza;
- a ogni checkpoint si misura il verdetto su `dev_select`; un checkpoint si accetta solo se BAcc resta entro 0,3 punti ed ECE entro 0,01 da quello del piano binario;
- se la calibrazione si sposta, si aggiunge un termine che tiene le probabilità vicine a quelle del verdetto di partenza.

```
gpu-run --batch --name ev-joint-knowline -- bash -c \
  'python -m evidence.train_joint --model knowline-4b --mix ev_joint --config configs/<migliore>.json \
     --lambda 0.3 --run joint > logs/ev-joint-knowline.out 2>&1'
```

Passo 10 — Valutazione

In breve

Per ogni modello si misurano tre cose:

1. il **verdetto** non è cambiato;
2. le **frasi mostrate** sono quelle giuste, anche quando sono lontane fra loro o dal claim;
3. la **latenza** resta quella di un decision model.

Script: `evidence/eval_evidence.py` (qualità) ed `evidence/latency.py` (velocità).

1. Su quali righe

L'evidenza si misura sulle righe supportate **che il modello giudica supportate**: sono le sole in cui il sistema mostra frasi. A parte si riporta quante righe supportate il modello giudica non supportate (e quindi restano senza evidenza).

Per confrontare i modelli si usano anche le sole righe giudicate supportate da tutti.

2. Metriche dell'evidenza

Metrica	Che cosa misura
Precisione, recall, F1 per frase	quante frasi mostrate sono giuste, quante giuste sono state mostrate
Insieme completo	in quante righe sono state mostrate **tutte** le frasi di evidenza (con più gruppi alternativi, basta uno). Per l'evidenza su più frasi è la metrica che conta: metà evidenza non convince
Corrispondenza esatta	frasi mostrate = frasi di evidenza
AUPRC	quanto bene i punteggi ordinano le frasi, indipendentemente dalla soglia
Errore sul numero	di quante frasi in più o in meno rispetto all'evidenza
F1 per distanza fra frasi	vicine / 2–9 / 10–19 / 20 o più frasi fra la prima e l'ultima frase di evidenza
F1 per distanza dal preambolo	ultima frase di evidenza a meno di 1.024 / 1.024–4.096 / oltre 4.096 token dal claim del preambolo
Sufficienza	in quante righe il documento ridotto alle sole frasi mostrate fa ancora dire "supportato" al modello. Non dipende dalle annotazioni, che possono essere incomplete
Necessità	in quante righe il documento senza le frasi mostrate fa dire "non supportato" (si riporta solo: se l'evidenza è ripetuta nel documento può fallire anche quando è giusta)
Negativi accesi	su `dev_evid_neg`: in quanti claim non supportati almeno una frase supera la soglia (non viene mostrata, ma indica che la testa si fa ingannare dalla somiglianza)

Sufficienza e necessità richiedono 2 forward per riga: si calcolano solo in valutazione, con lo stesso modello e lo stesso prompt di produzione.

3. Metriche del verdetto

Le stesse del piano binario (BAcc, AUC, ECE) su `dev_select`, test e AggreFact, con il prompt con preambolo. Nelle fasi 1a e 1b devono coincidere con quelle del verdetto scelto al passo 05: se no, c'è un errore nella pipeline.

4. Prove di comportamento

Costruite dal test con le trasformazioni del passo 03:

Prova	Righe	Risultato atteso
Evidenza spostata all'inizio / a metà / alla fine	3 × 150	stesse frasi mostrate in ≥ 90% delle righe
Evidenza spostata oltre 1.024 e oltre 4.096 token	2 × 150	idem
Frasi di disturbo aggiunte	200	nessuna frase di disturbo mostrata in ≥ 95%
Frase di evidenza ripetuta altrove	150	almeno una delle due copie mostrata, sempre
Frase di evidenza riscritta senza parole in comune col claim	150	mostrata in ≥ 80%
Coppia quasi identica: positivo e negativo con una frase tolta	200 coppie	positivo: frasi giuste; negativo: "non supportato" e nessuna frase sopra soglia

5. Latenza

Stesso server (12), stessa GPU, 1.000 richieste reali di lunghezze diverse (le prime 50 non si contano). Tre configurazioni:

	Prompt	Uscita
A	senza preambolo	solo verdetto (come il piano binario)
B	con preambolo	solo verdetto
C	con preambolo	verdetto + evidenza

Obiettivo: C al massimo +3% rispetto a B (modelli 4B: +3% o +2 ms). La differenza fra B e A è il costo del preambolo e si riporta a parte. Si misura anche il tempo della sola testa, per sapere dove intervenire se si supera il limite.

6. Comandi

```
# durante gli esperimenti
python -m evidence.eval_evidence --model knowline-4b --run data/evidence/runs/knowline-4b/<id> --block
↪  dev_evid_select

# test: una volta sola, a configurazione congelata (passo 12)
gpu-run --batch --name ev-test-knowline -- bash -c \
  'python -m evidence.eval_evidence --model knowline-4b --release <rilascio> \
     --block test_evid_id test_evid_ood test_evid_sparse --sufficiency > logs/ev-test-knowline.out 2>&1'
gpu-run --batch --name ev-latency-knowline -- bash -c \
  'python -m evidence.latency --model knowline-4b --release <rilascio> > logs/ev-latency-knowline.out 2>&1'
```

Lo script salva `metrics.json` e `predictions.jsonl` (per ogni riga: punteggi, soglia, frasi mostrate, frasi di evidenza), così le metriche si ricalcolano senza rifare i forward.

7. Report finale (uno per modello)

Set	Metrica	E-LEX	E-ATT	E-ABL	Testa
test_evid_id	F1 / insieme completo				
test_evid_ood	F1 / insieme completo				

Set	Metrica	E-LEX	E-ATT	E-ABL	Testa
test_evid_sparse	F1 a ≥ 20 frasi / oltre 4.096 token				
tutti	sufficienza				
dev_evid_neg	negativi accesi				
verdetto	BAcc (uguale al piano binario?)	—	—	—	
latenza	p50 / p95, C contro B				

Passo 11 — Esperimenti

In breve

Si parte da una testa semplice (H0) e si aggiunge **un pezzo alla volta**, tenendolo solo se migliora. Tutto si fa su **KnowLine-4B** con i vettori salvati (fase 1a): ogni prova è un training della sola testa. Alla fine elephant-1 e gli altri decision model ripetono la configurazione migliore.

Regole

1. Una modifica per prova, con il suo file `configs/<id>.json`.
2. Si decide su `dev_evid_select` guardando quattro numeri: F1, insieme completo, F1 con evidenza a 10 o più frasi di distanza, F1 con evidenza oltre 1.024 token. In più, i negativi accesi su `dev_evid_neg`.
3. **Una modifica si tiene** se aumenta l'F1 di almeno 1 punto o l'insieme completo di almeno 2, senza far scendere gli altri numeri di più di 1 punto. Se aggiunge dati da leggere dal modello, deve anche restare nel limite di latenza.
4. A parità di risultato si tiene la versione più semplice.
5. Se il guadagno è sotto i 2 punti, si ripete con 2 semi in più prima di tenerlo.

Prima degli esperimenti

Che cosa	Dove
Testo del preambolo e verdetto da usare per ogni modello	05
Metodi semplici di confronto (E-LEX, E-ATT, E-ABL, …)	07

La testa di partenza: H0

Solo l'ultimo layer, solo il vettore dell'ultimo token di ogni frase, niente attenzione, 2 layer di transformer fra le frasi, loss di partenza con soglia fissa, solo righe positive.

Prima domanda: H0 batte E-LEX ed E-ATT? Se no, si cerca un errore nelle posizioni o nei vettori (06 §6) prima di provare altro.

Gli esperimenti principali (in quest'ordine)

Nome	Che cosa si cambia	Che cosa si guarda
H-LOSS	loss con soglia per riga invece di soglia fissa (08 §3)	insieme completo; numero di frasi mostrate
H-MIL	si aggiungono i claim non supportati al training (nessuna frase sopra soglia), con peso 0,3 e 1	negativi accesi; che non si perdano frasi nei positivi incerti

Nome	Che cosa si cambia	Che cosa si guarda
H2a	vettori da 4 layer invece che dal solo ultimo; poi solo i 2 più usati	F1; quali layer pesano
H2b	si aggiunge l'attenzione ricevuta da ogni frase	F1 con evidenza lontana
H2c	si aggiunge la media dei token della frase	F1 sulle frasi lunghe (oltre 25 parole)

Controlli sulla configurazione migliore

Ognuno toglie o cambia un pezzo, per capire quanto conta:

Nome	Che cosa si cambia	Che cosa dice
H3a	niente transformer fra le frasi: ogni frase ha un punteggio da sola	se le frasi devono comunicare. Atteso: crollo su HoVer e sull'evidenza lontana. Se non crolla, si può semplificare
H3b	punteggio = somiglianza fra `x_0` e la frase	se basta una testa più semplice
H3c	1 o 4 layer di transformer invece di 2	quanta capacità serve
H4	vettori calcolati senza preambolo	quanto aiuta il preambolo. Se poco, si toglie
H5	niente righe con evidenza spostata nel training	se servono. Atteso: calo sull'evidenza lontana
H7	peso 1 invece di 0,3 per le frasi non segnate di FEVER; per FEVER basta un gruppo di evidenza invece di tutti	come trattare annotazioni incomplete
H8	solo evidenza annotata / + dati generati / + etichette ricavate col modello / + positivi senza evidenza (al posto delle etichette ricavate)	quanto valgono i dati del passo 3
H9	niente informazione sulla posizione della frase	se la testa preferisce troppo l'inizio o la fine

Esperimenti da fare solo se serve

Nome	Quando	Che cosa
H2d	se H2c ha aiutato sulle frasi lunghe	media pesata dei token decisa dalla testa (fase 1b, 09 §4)
H-PAIR	se l'insieme completo su HoVer resta sotto 0,65, o l'F1 cala di oltre 5 punti con frasi lontane	punteggio per coppie di frasi (08 §4)
H6	se gli obiettivi non sono raggiunti dopo tutto il resto	fase 2, con peso dell'evidenza 0,1 / 0,3 / 1

elephant-1 e gli altri modelli

Nome	Che cosa
E0	elephant-1: si salvano i vettori e si addestra la configurazione migliore di KnowLine; controllo rapido di quali layer usare (possono essere diversi)
H4-bi	H4 su elephant-1: con l'attenzione bidirezionale il preambolo serve ancora? Se no, per i modelli bidirezionali si toglie
X0	gli altri decision model, con la configurazione migliore del loro gruppo di attenzione (causale: KnowLine; bidirezionale: elephant-1), nell'ordine del piano binario (prima Jeffan-4B, d3-flash, OneJev, Nimble; poi StartLux, pplx, Clef)
E6	fase 2, solo se su KnowLine ha dato un guadagno chiaro e il modello è sotto gli obiettivi
L1	latenza per ogni modello rilasciato (10 §5)

Controlli finali

Nome	Che cosa
C1	3 semi della testa finale per modello: media e variabilità
C2	sufficienza e necessità su tutti i set di test
C3	testa contro E-ABL: quanto ci si avvicina con un forward a un metodo con centinaia di forward
C4	confronto fra i modelli sulle righe giudicate supportate da tutti

Registro

`data/evidence/runs/registry.md`, una riga per prova (scritta in automatico da `train_head.py`, tranne la decisione):

Nome	Modello	Fase	Modifica	F1	Insieme completo	F1 ≥ 10 frasi	F1 > 1.024 token	Negativi accesi	Sufficienza	Δ verdetto	Δ latenza	Decisione
E-LEX	—	—	—							—	—	riferimento
H0	KnowLine-4B	1	—							0		

Passo 12 — Server e rilascio

In breve

Per ogni modello si prepara una cartella di rilascio e un server (`evidence/serve.py`) che, per ogni richiesta, fa **un forward** e restituisce verdetto ed evidenza. Il server usa lo stesso codice della valutazione (divisione in frasi, prompt, hook): così quello che si è misurato è quello che gira.

1. Che cosa fa il server per ogni richiesta

```
richiesta {document, claim}
  1. divide il documento in frasi (sentences.py)
```

```
  2. costruisce il prompt con il preambolo e trova i token di ogni frase (render.py)
  3. un forward del modello con gli hook → punteggi "sì"/"no", vettori delle frasi, attenzione
  4. P(supportato); se non supera la soglia → risposta senza evidenza
  5. testa di evidenza → punteggi delle frasi e soglia della riga
  6. frasi sopra la soglia (almeno 1, al massimo 5) → posizioni nel documento originale
```

```python
def answer(document, claim):
    spans = sentences.split(document)
    ex = render(document, claim, spans, preamble=DECISION["preamble"])
    logits, feats = model.forward(ex)
    p = p_supported(logits, DECISION["temperature"])
    if p <= DECISION["threshold"]:
        return {"p_supported": p, "supported": False, "evidence": []}
    s, s0, _ = head(**feats)
    margin = s - s0 - DECISION["evidence"]["delta"]
    order = margin.argsort(descending=True).tolist()
    chosen = [k for k in order if margin[k] > 0][: DECISION["evidence"]["max_evidence"]] or order[:1]
    return {"p_supported": p, "supported": True,
            "evidence": [{"sentence": k, "start": spans[k][0], "end": spans[k][1],
                          "score": float(torch.sigmoid(s[k] - s0))} for k in sorted(chosen)]}
```

`start`/`end` si riferiscono al documento come l'ha inviato il client, non al prompt.

2. Come gira ogni modello

Il server ha bisogno degli **stati interni** del forward, non solo della risposta: serve un forward in transformers con gli hook di 01. Quindi:

Modello	Server per verdetto + evidenza
KnowLine-4B	front end degli autori con backend transformers (`hf`), più gli hook
elephant-1	motore `qev` in-process (transformers), più gli hook
Jeffan-4B, d3-flash, pplx-decider-flash, Clef-Flash	il codice degli autori in transformers, più gli hook
OneJev1.6-9B, Nimble-9B-v3	transformers con il prompt degli autori (il loro runtime, Flide o il motore Nimble, non espone gli stati); verdetto verificato contro il runtime degli autori
StartLux 9B/4B	transformers con la maschera bidirezionale ricostruita (il server degli autori è vLLM e non espone gli stati); verdetto verificato contro il loro server

Per tutti servono `flash-linear-attention` e `causal-conv1d` (layer DeltaNet), altrimenti il forward è molto più lento.

Regole:

- dalla GPU escono solo i vettori delle frasi (o direttamente i punteggi), mai quelli di tutti i token;
- richieste in batch raggruppate per lunghezza simile;
- documenti oltre 32.768 token: errore esplicito (`document_too_long`), mai tagliati in silenzio;
- all'avvio il server controlla che i file caricati abbiano gli hash registrati in `frozen.json`.

3. La cartella di rilascio

`releases/<modello>-fc-ev-<versione>/`:

File	Contenuto
pesi del verdetto	quelli del piano binario (o riaddestrati col preambolo, o con la fase 2)

File	Contenuto
`evidence_head.safetensors`	pesi della testa (~10–20 MB)
`decision.json`	tutte le impostazioni (sotto)
`frozen.json`	hash di pesi, testa, codice di divisione in frasi e di rendering, `question.json`, `decision.json`, mix, split
`report.md`	risultati di test e latenza (10 §7)
`MODEL_CARD.md`	descrizione per chi usa il modello

```
{"model": "knowline-4b-fc-ev-v1",
 "question_id": "unsupported_detail", "take": "no",
 "preamble": "Claim under review: {claim}",
 "temperature": 1.0, "threshold": 0.50,
 "evidence": {"mix_layers": [19, 23, 27, 31], "attention_layers": [3, 7, 11, 15, 19, 23, 27, 31],
              "features": ["end", "mean", "att"], "head_config": "configs/<id>.json",
              "delta": 0.0, "max_evidence": 5, "min_evidence": 1},
 "segmenter": "evidence/sentences.py@<sha256>",
 "max_document_tokens": 32768}
```

(Valori di esempio.) Se cambia la divisione in frasi, il preambolo o i layer, la testa va riaddestrata.

4. Controlli prima del rilascio

Controllo	Deve dare
Verdetto del server = verdetto in valutazione (200 righe)	differenza media di P < 0,005
Evidenza del server = evidenza in valutazione (200 righe)	stesse frasi in almeno 199
Richieste in batch con lunghezze diverse	stesse frasi che una alla volta
Documenti con ", \, a capo, <, emoji, testo non latino	il testo fra `start` e `end` è la frase giusta
Latenza	entro il limite (10 §5)

5. Congelamento e test

1. Scrivere `frozen.json`.
2. Eseguire **una sola volta** i set di test e la misura di latenza; scrivere `report.md`.
3. Model card: verifica documento–claim in inglese; evidenzia frasi **solo** quando il claim è supportato; non dice cosa manca né cosa contraddice; il prompt contiene una riga col claim prima del documento; documenti fino a 32.768 token.
4. Licenze: quelle dei pesi di partenza (MIT per elephant-1; Apache 2.0 per KnowLine-4B, Jeffan-4B, d3-flash, OneJev, pplx-decider-flash, Clef-Flash; **CC BY-NC per StartLux e Nimble**: derivati non commerciali). Dati usati per la testa: FEVER e HoVer CC BY-SA.

Checklist e problemi frequenti

1. Checklist per passo

Passo 1 — Modelli

- ☐ Pacchetti per i layer DeltaNet installati (`flash-linear-attention`, `causal-conv1d`).
- ☐ I modelli si caricano con la funzione di attenzione che registra i pesi, senza perdere la loro maschera.
- ☐ Test superati: verdetto invariato, layer giusti, attenzione uguale alla modalità “eager”, memoria.

Passo 2 — Dati dalle fonti

- ☐ `sentences.py` scritto e fissato.
- ☐ FEVER: indice delle pagine, testo rimesso in forma normale, train/dev/test.
- ☐ HoVer: `hotpot_qa` scaricato, documenti con pagine di disturbo, evidenza allineata (100 pagine controllate).
- ☐ SciFact e WiCE pronti come test.
- ☐ Almeno il 90% delle frasi di evidenza originali corrisponde a una nostra frase; 30 righe per fonte lette.

Passo 3 — Altri dati

- ☐ C2D con posizioni (con negativi e claim parziali); CG2C con la frase di ogni fatto.
- ☐ Etichette ricavate con KnowLine-4B: giudice d'accordo ≥ 80%, accordo con elephant-1 ≥ 0,7.
- ☐ Righe con evidenza spostata, senza riferimenti rotti.

Passo 4 — Split e mix

- ☐ Set di valutazione congelati, con le distribuzioni delle distanze.
- ☐ Mix `ev_head` che rispetta tutti i vincoli.

Passo 5 — Preambolo

- ☐ Testo scelto; per ogni modello deciso se il verdetto si riusa o si riaddestra; `question.json` aggiornato.

Passo 6 — Token e vettori

- ☐ Posizioni dei token giuste (200 su 200), anche con caratteri speciali.
- ☐ Vettori di KnowLine-4B salvati per il mix e per i set dev.

Passo 7 — Metodi semplici

- ☐ E-RAND, E-LEX, E-EMB, E-ATT su `dev_evid_select`; E-ABL su 300 righe per fonte.

Passo 8 — Testa e loss

- ☐ Test automatici superati.

Passo 9 — Training

- ☐ La testa impara a memoria 64 righe; H0 riproducibile.
- ☐ Regola per le frasi mostrate scelta e confermata.

Passo 10 — Valutazione

- ☐ Tutte le metriche, le prove di comportamento e la latenza A/B/C.

Passo 11 — Esperimenti

- ☐ Esperimenti principali e controlli su KnowLine-4B; configurazione ripetuta su elephant-1 e sugli altri modelli; registro compilato.

Passo 12 — Rilascio

- ☐ Controlli del server superati; `frozen.json` scritto prima dei test; test eseguiti una volta; model card.

2. Se succede...

Problema	Prima cosa da controllare	Poi
La testa non impara (F1 vicino al caso)	le posizioni dei token di ogni frase (passo 6, test)	le impostazioni dei vettori salvati (`meta.json`)
La testa fa come E-LEX: sbaglia frasi riscritte e frasi ponte	quante righe HoVer, CG2C, C2D ci sono nel mix	più negativi con frasi simili al claim; H-MIL con peso 1
Bene su FEVER, male su HoVer e con frasi lontane	H3a: il transformer fra le frasi serve?	più righe spostate; H-PAIR; 4 layer

Problema	Prima cosa da controllare	Poi
Male quando l'evidenza è lontana dal preambolo	quante righe lontane ci sono nel mix	attenzione come feature (H2b); fase 2
Mostra troppe frasi	la regola di selezione (δ) e i negativi nel training	peso più alto per le frasi non di evidenza
Non mostra quasi nulla sui positivi incerti	peso dei negativi	abbassarlo
Mostra spesso la prima frase o il titolo	dove stanno le frasi di evidenza nel mix	H9 (senza posizione)
Precisione bassa ma sufficienza alta	leggere a mano le frasi "sbagliate": spesso sono evidenza valida non annotata	peso più basso per le frasi non segnate di quella fonte
Le etichette ricavate col modello peggiorano (H8)	accordo del giudice e fra KnowLine ed elephant-1	usare solo le righe concordi, o i positivi senza evidenza
Risultati molto diversi fra semi	se la testa dipende da poche teste di attenzione	più dropout sull'attenzione, o toglierla
Il verdetto cambia nelle fasi 1a/1b	non deve succedere: il prompt è diverso da quello del passo 5	test di identità del prompt
Il verdetto peggiora nella fase 2	learning rate della LoRA, peso dell'evidenza	termine che tiene il verdetto vicino a quello di partenza; tornare alla fase 1
Troppo lento	tempo della testa misurato a parte	meno layer, togliere media o attenzione, 1 layer di transformer
Estrazione o server molto lenti	i pacchetti per DeltaNet sono davvero usati?	batch per lunghezza, più job
Spazio su disco insufficiente	quanti layer si salvano	solo i layer utili su KnowLine

Una strada da esplorare: insegnare alla testa quanto ogni frase regge il verdetto

Il problema

Le frasi di evidenza annotate vengono quasi tutte da Wikipedia (FEVER, HoVer). In produzione arrivano altri testi: riassunti, risposte di sistemi RAG, notizie, documenti tecnici. Su questi abbiamo il verdetto ma non sappiamo quali frasi sono l'evidenza, quindi la testa non impara a lavorarci.

Le etichette ricavate col modello (passo 03 §2) aiutano solo in parte: dicono quali frasi bastano, ma non quanto conta ciascuna, e costano una ricerca per riga.

L'idea

Su **qualunque** riga supportata, di qualunque tipo di testo, si può misurare senza annotazioni quanto ogni frase regge il verdetto: si toglie la frase dal documento e si guarda di quanto scende il punteggio del verdetto.

```
z            = punteggio del verdetto sul documento intero
z senza k    = punteggio col documento senza la frase k        (un forward per frase)
importanza_k = z − z senza k                                     quanto la frase k regge il verdetto
```

La testa impara a prevedere questo ordinamento in un solo forward. Così impara, sui testi di produzione, quali frasi reggono davvero il verdetto, che è anche la metrica di sufficienza.

Se la stessa informazione è ripetuta in due frasi, togliendone una il verdetto non cambia, e entrambe sembrano poco importanti. Per questo si tiene anche il gruppo trovato con l'ablazione del passo 3, che gestisce questo caso.

Come farlo

1. **Righe**: i positivi del training del piano binario non Wikipedia (C2D, D2C, SummEdits, AVeriTeC e le altre fonti), più FEVER e HoVer per confrontare con le annotazioni. Niente righe di test né fonti di AggreFact.
2. **Calcolo con KnowLine-4B** (`evidence/label_counterfactual.py`): per ogni riga, un forward per frase tolta. KnowLine è causale, quindi i prompt sono uguali fino alla frase tolta e si riusa il calcolo della parte comune (con un modello bidirezionale non si potrebbe). Documenti fino a 80 frasi; oltre, solo per le 50 frasi più promettenti.
3. **Prima verifica, senza training**: su `dev_evid_select`, si usa l'importanza come punteggio delle frasi e si confronta con le annotazioni. Se non fa meglio di E-LEX, il segnale è troppo rumoroso e ci si ferma.
4. **Accordo fra modelli**: su 300 righe si rifà il calcolo con elephant-1. Se gli ordinamenti sono molto diversi, per gli altri modelli si ricalcola col modello stesso.
5. **Training** (fase 1a, esperimento H-CF): alla loss della testa si aggiunge un termine che avvicina l'ordinamento dei punteggi della testa a quello dell'importanza (KL fra le due distribuzioni normalizzate sulle frasi), con peso 0,3 e 1. Il gruppo trovato con l'ablazione fa da evidenza, con peso 0,5.
6. **Valutazione** su tre set:
   - `dev_evid_select` (Wikipedia): non deve peggiorare di più di 1 punto di F1;
   - `test_evid_ood` (SciFact, WiCE);
   - un nuovo piccolo set di 200 righe di riassunti e risposte RAG dal dev del piano binario, con evidenza indicata dal giudice e controllata a mano.

Quando tenerlo

Rispetto alla configurazione migliore senza questa aggiunta:

- sufficienza almeno +3 punti su tutti i set;
- F1 almeno +2 punti sul set di riassunti e RAG;
- F1 su `dev_evid_select` non più di 1 punto sotto.

Rischi

Rischio	Cosa fare
L'importanza segue le scorciatoie del modello (per esempio frasi con parole del claim che il modello sopravvaluta)	la verifica del punto 3; tenere sempre le annotazioni dove ci sono
Informazione ripetuta: importanza bassa per tutte le copie	il gruppo dell'ablazione come evidenza
Costo: un forward per frase	KnowLine-4B, riuso della parte comune, limite di 80 frasi
Se il modello viene riaddestrato (fase 2), l'importanza calcolata prima non vale più	usarla solo col modello congelato, o ricalcolarla

Perché vale la pena

È l'unico modo, fra quelli del piano, di avere un segnale di evidenza sui testi di produzione senza annotarli a mano, ed è legato direttamente a ciò che interessa all'utente: che le frasi mostrate bastino a reggere il verdetto. Il costo è tutto prima del rilascio e riusa codice già previsto; in produzione non cambia nulla.